Sapienza
Università di Roma

Experimental Evaluation of a Multi-Layer Abstraction Approach for RL on Non-Markovian Tasks

Ingegneria Informatica, Automatica E Gestionale "ANTONIO RUBERTI"
Engineering in Computer Science

Matteo Ventali
ID number 1985026

Advisor
Prof. Fabio Patrizi

Academic Year 2025/2026

To my mother, my guiding light in life.
A mia mamma, la mia guida nella vita.

Abstract

Applying Reinforcement Learning to areas involving sparse rewards and objectives that extend over time could result prohibitive. Useful feedback can be delayed and the present state of the environment may not be enough to show how far along the task is. Until now, in order to reduce the complexity of the learning problem, several methods based on Hierarchical Reinforcement Learning and state abstraction mechanism have been already suggested in the literature.

In this thesis we look at a multi-level abstraction framework that makes use of a hierarchy of ever coarser Markov Decision Processes in order to obtain value-based shaping signals which guide learning at the lower levels. Starting from this architecture, first of all, the framework is extended to handle non-Markovian tasks that are given in LTL_f and are automatically converted into Deterministic Finite Automata, thereby allowing temporal progress towards the task to be explicitly represented across the entire abstraction hierarchy.

Then, the full implementation of the proposed approach is tested on a complex domain considering non-Markovian task specifications. In particular, the experimental study focuses on episodic tasks that involve increasing temporal complexity, various abstraction configurations, and cyclic temporal behaviours. The complex domain considered suitable to carry out this empirical evaluation is the well-known *Gymnasium Lunar Lander*. The results are employed in order to evaluate the effectiveness of abstraction-based guidance in this context and to determine the main practical limitations that arise when the framework is applied to complex non-Markovian learning problems.

Contents

Chapter 1

Introduction

1.1 Motivation and Context

1.2 Problem Statement

1.3 Research Objectives and Questions

1.4 Contributions

1.5 Scope and Assumptions

1.6 Thesis Organization

Chapter 2

Background

2.1 Reinforcement Learning

Within the Machine Learning area, a general distinction is made between supervised, unsupervised, and Reinforcement Learning. Reinforcement Learning (RL) is a branch of Machine Learning (ML) concerned with learning how to make sequential decisions through interaction with an environment.

In *supervised learning*, an algorithm is trained on a labeled dataset containing examples, with the objective of learning a mapping that generalizes to samples not belonging to it. On the other hand, *unsupervised learning* works on unlabeled data and aims to identify underlying structures and patterns within the available observations [13].

Reinforcement Learning differs from both paradigms because the learner is not provided with explicit examples of the correct action to perform. Instead, an agent interacts with an (initially) unknown environment, selects actions, observes their consequences, and receives a reward signal that provides feedback about its behavior. Therefore, the goal of the agent is to learn a policy that maximizes the expected cumulative reward over time.

This type of interaction describes a *sequential decision-making problem*. Unlike prediction problems in which individual outputs can often be considered independently, in RL the consequences of a decision may extend beyond the immediate transition. An action selected at a given time may influence the state reached by the agent, the actions that will subsequently become available, and the rewards that can be obtained in the future. So, the quality of a decision cannot generally be evaluated only in terms of its immediate outcome, but must also account for its long-term consequences. In fact, in RL we often refer to trajectories (sequences of states and actions) generated by the agent's behavior during its interaction with the environment.

RL can be viewed as a framework for sequential decision-making under uncertainty, where an agent must learn how its actions influence both immediate and future outcomes. To reason formally about these interactions, a mathematical model of the decision process is required. The standard framework adopted for this purpose is the Markov Decision Process (MDP), which provides a formal description of states, actions, transition dynamics, and rewards. MDPs are introduced in the following section and will be the basis for the RL methods discussed throughout this chapter.

2.1.1 Markov Decision Processes

The mathematical framework adopted for modeling sequential decision-making problems is the Markov Decision Process (MDP). An MPD is defined as a tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle \tag{2.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P is the transition probability function, R is the reward function, and $\gamma \in [0, 1]$ is the discount factor [13].
At each discrete time step t, the environment is in a state $S_t \in \mathcal{S}$. Based on this state, the agent selects an action $A_t \in \mathcal{A}$. The execution of the action triggers the environment to transition into a new state S_{t+1} and produces a scalar reward R_{t+1}.

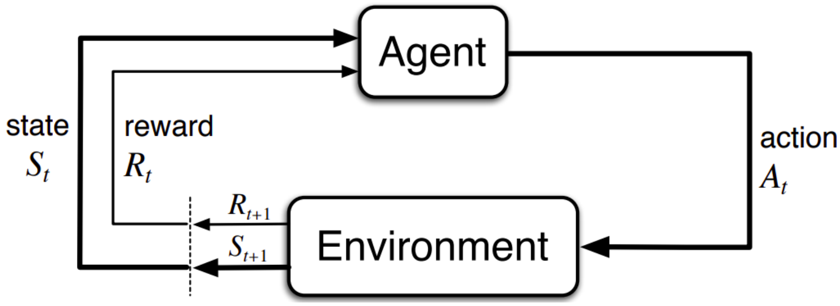

Figure 2.1. Interaction between the agent and the environment in an MDP [13].

The transition dynamics are characterized by

$$P(s' \mid s, a) = \Pr\big(S_{t+1} = s' \mid S_t = s, A_t = a\big) \tag{2.2}$$

which denotes the probability of reaching state s' after executing action a in state s. Looking at the P definition, it is immediate to notice that s' strictly depends only on the immediate previous state s. Therefore, with this definition, we are implicitly assuming the so-called *Markov property*: conditioned on the current state and action, the distribution of the next state is independent of the preceding interaction history. Formally, it can be expressed as follow:

$$\Pr\big(s' \mid s, a, S_{t-1}, A_{t-1}, \ldots, S_0\big) = \Pr\big(s' \mid s, a\big) \tag{2.3}$$

The current state must contain all the information relevant to predicting the future evolution of the process [13]. If additional information from the interaction history is required, the chosen state representation is *not Markovian* with respect to the problem under consideration.
The reward function assigns a numerical signal to the agent's interaction with the environment. The most general formulation corresponds to the function $R(s, a, s')$ which provides immediate feedback, whereas the agent's objective generally depends on rewards accumulated over multiple time steps. The distinction between immediate and long-term consequences is therefore central to sequential decision-making.

The discount factor γ determines the relative importance of future rewards. Values close to zero emphasize immediate outcomes, whereas values close to one assign greater importance to rewards received further in the future. The accumulation of discounted rewards will be formalized through the concept of return in Section 2.1.2. A sequence generated through interaction between the agent and the environment can be written as

$$s_0, a_0, r_1, s_1, a_1, r_2, s_2, \ldots \tag{2.4}$$

and is referred to as a *trajectory*. The probability of a trajectory is determined jointly by the transition dynamics of the MDP and the actions chosen by the agent based on its own *policy*.

2.1.2 Policies, Returns, and Value Functions

A *policy* specifies how the agent selects actions in each state. A stochastic policy π assigns a probability to every action $a \in \mathcal{A}$ given a state $s \in \mathcal{S}$:

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s) \tag{2.5}$$

For every state, these probabilities satisfy $\pi(a \mid s) \geq 0$ and $\sum_{a \in \mathcal{A}} \pi(a \mid s) = 1$. A deterministic policy is the special case in which one action is selected with probability one and can therefore be represented as a mapping $\pi : \mathcal{S} \to \mathcal{A}$.
The immediate reward obtained from a single transition is generally insufficient to evaluate an action, since its consequences may affect rewards received many steps later. The long-term outcome from time step t is represented by the *return*, defined as the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \tag{2.6}$$

In episodic tasks, this sum terminates when a terminal state is reached. The discount factor γ controls the relative importance of immediate and delayed rewards. For continuing tasks with bounded rewards, choosing $\gamma < 1$ also ensures that the infinite sum remains finite.
Value functions measure the expected return obtained when the agent follows a given policy. The *state-value function* evaluates a state, whereas the *action-value function* evaluates the selection of a particular action in that state:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \tag{2.7}$$
$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \tag{2.8}$$

The expectation accounts for both the actions selected according to π and the stochastic transitions of the environment. The state-value and action-value functions are related by

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, Q^\pi(s, a) \tag{2.9}$$

An optimal policy π^* maximizes the expected return from every state. The corresponding optimal value functions are

$$V^*(s) = \max_{\pi} V^{\pi}(s) \tag{2.10}$$

$$Q^*(s,a) = \max_{\pi} Q^{\pi}(s,a) \tag{2.11}$$

Once Q^* is known, an optimal deterministic policy can be obtained by selecting an action with maximum value:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} Q^*(s,a) \tag{2.12}$$

Policies and value functions therefore provide the principal quantities used to describe and compare decision-making strategies. Their recursive structure is captured by the Bellman equations [13].

2.1.3 Bellman Equations and Value Iteration

The return in Equation (2.6) can be decomposed into the immediate reward and the discounted return from the next time step:

$$G_t = R_{t+1} + \gamma G_{t+1} \tag{2.13}$$

This recursive relation leads to the *Bellman expectation equation*. For a policy π, the value of a state equals the expected immediate reward plus the discounted value of the successor state:

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^{\pi}(s')\right] \tag{2.14}$$

The corresponding equation for the action-value function is

$$Q^{\pi}(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q^{\pi}(s',a')\right] \tag{2.15}$$

These equations express a consistency condition: the value assigned to the current state or action must agree with the values assigned to its possible successors. For an optimal policy, the expectation over the next action is replaced by a maximization, yielding the Bellman optimality equations

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^*(s')\right] \tag{2.16}$$

$$Q^*(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \max_{a' \in \mathcal{A}} Q^*(s',a')\right] \tag{2.17}$$

When the transition and reward models are known, the optimal state-value function can be computed through *value iteration*. Starting from an arbitrary estimate V_0, the Bellman optimality backup is applied repeatedly:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma V_k(s') \right] \tag{2.18}$$

For a finite discounted MDP, the sequence of estimates converges to V^*. An optimal policy can then be extracted by acting greedily with respect to the converged values:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^*(s') \right] \tag{2.19}$$

Value iteration is a model-based planning method because it requires explicit knowledge of P and R. In RL, these quantities are unknown, motivating the adoption of model-free methods like Q-learning [13].

Policy Evaluation

Given a fixed policy π, the associated V^π can be computed by applying the Bellman expectation equation in an iterative way. Starting from a random initialization V_0, the update is

$$V^\pi_{k+1}(s) = \sum_{s' \in \mathcal{S}} P(s' \mid s, \pi(s)) \left[R(s, \pi(s), s') + \gamma V^\pi_k(s') \right] \tag{2.20}$$

for a deterministic policy π. The procedure is repeated until the value function converges. For finite discounted MDPs, iterative policy evaluation converges to V^π.

2.1.4 Value-Based Methods

Value-based RL methods compute an optimal policy by estimating value functions rather than representing the policy π directly. In particular, action-value methods learn the expected return associated with each state-action pair and derive a policy by selecting actions adopting a greedy strategy with respect to these estimates. These algorithms can be classified in two different categories:

- *on-policy*: the algorithm learns the values of the current policy. In other words, the data used for the policy update are generated adopting the same policy the algorithm is trying to improve.
- *off-policy*: the algorithm learns the values adopting a different policy with respect to the one is trying to improve. We usually refer to the latter as *Behavior Policy*, which controls directly the exploration and interaction with the environment during the learning phase.

Q-learning. Q-learning is a model-free and off-policy algorithm that estimates the optimal action-value function directly from observed transitions [16]. After observing the transition $(S_t, A_t, R_{t+1}, S_{t+1})$, it applies the temporal-difference update

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q_t(S_{t+1}, a') - Q_t(S_t, A_t) \right] \tag{2.21}$$

where α is the learning rate hyper-parameter. The maximization defines a greedy target policy, while experience can be collected through an exploratory behavior policy such as ϵ-greedy. The most straightforward implementation of it is Tabular Q-learning which is particularly effective for small discrete MDPs, but it becomes impractical when the state space is large because it maintains a separate estimate for every (s, a) pair [13].

Deep Q-Networks. Deep Q-Networks (DQN) address the limitation of Q-learning by approximating the action-value function with a neural network $Q(s, a; \theta)$. To improve training stability, DQN stores transitions in a replay memory and learns from randomly sampled mini-batches. It also maintains a target network with parameters θ^-, which are periodically synchronized with the online parameters θ. For a transition (s, a, r, s', d), the target is

$$y^{\mathrm{DQN}} = r + \gamma(1-d) \max_{a' \in \mathcal{A}} Q(s', a'; \theta^-) \tag{2.22}$$

where d indicates whether the transition is terminal. The online network is trained by minimizing the squared difference between this target and $Q(s, a; \theta)$ [9].

Double Deep Q-Networks. The maximization in the DQN target can lead to overestimated action values because the same estimates are used both to select and evaluate the next action. Double Deep Q-Networks (DDQN) reduce this bias by using the online network for action selection and the target network for action evaluation:

$$y^{\mathrm{DDQN}} = r + \gamma(1-d) Q\left(s', \arg\max_{a' \in \mathcal{A}} Q(s', a'; \theta); \theta^-\right) \tag{2.23}$$

DDQN retains the replay memory, target network, and optimization procedure of DQN while changing only the construction of the temporal-difference target. This generally produces more reliable value estimates. [15].

2.2 Non-Markovianity and Temporally Extended Tasks

In several situations, a problem specification cannot be expressed as an objective that depends only on the last state of the environment. Specifically, in this scenarios, a successful behavior could depend on the order in which events occur, on whether certain conditions have already been satisfied, or on whether specific events are eventually reached during the execution.

For instance, consider a task requiring an agent to first visit a region (A) and subsequently reach a region (B). Reaching (B) cannot be considered sufficient to determine task completion unless we have not already reached region (A) in the past. Therefore, two trajectories ending in the region (B) may correspond to different levels of progress with respect to the task.

This types of objective specifications are commonly referred to as *temporally extended tasks*, since their satisfaction is defined over sequences of states or observations rather than on individual states only. When the information required to evaluate the task is

not completely represented by the current environment state, the task introduces a form of *non-Markovian* dependence on the interaction history (the *Markov property* is violated).
Representing such objectives requires a formalism capable of expressing temporal relationships between events. In this thesis, we choose to express these goal specifications trough Linear Temporal Logic on finite traces (LTL_f), a logical language which allows temporal properties to be specified over finite executions. Then, LTL_f specifications can in turn be converted in a deterministic finite automata (DFA), whose states provide a way to monitor the task progress.

2.2.1 Markovian and Non-Markovian Tasks

As introduced in Section 2.1.1, the Markov property requires the current state to contain all the information needed to characterize the future evolution of the environment [13].
In a MDP, the probability distribution $P(s' \mid s, a)$ of the next state depends only on the current state and action, and not on the complete interaction history.
A similar distinction can be made with respect to the task or reward structure.
In a *Markovian task*, the relevance of a transition can be determined from the current state, action, and successor state. So, under this assumption, a reward function can be expressed as $R(s, a, s')$.

However, there are some objectives that cannot be evaluated using only the current transition. In particular, their satisfaction depends on events that have occurred earlier in the execution. Let

$$h_t = s_0, a_0, s_1, a_1, \ldots, s_t \tag{2.24}$$

denote the interaction history up to time t. When the reward or the task satisfaction depend on h_t, we are referring to a *Non-Markovian task specification*.
This distinction is crucial because the environment itself may still satisfy the Markov property even when the task specification does not. In other words, non-Markovianity may arise from the specification of the objective rather than from the dynamics of the environment. The physical state can therefore be sufficient to predict the evolution of the environment while being insufficient to determine the progress made toward the task.

This concept is formalized by the *Non-Markovian Reward Decision Process* (NMRDP) definition introduced in [3].
An NMRDP is defined as a tuple:

$$\mathcal{M}_{\text{NM}} = \left\langle \mathcal{S}, \mathcal{A}, P, \overline{R}, \gamma \right\rangle \tag{2.25}$$

where $\mathcal{S}$, $\mathcal{A}$, P, and γ have the same meaning as in an MDP, while the reward function is defined over finite sequences of state-action pairs:

$$\overline{R} : (\mathcal{S} \times \mathcal{A})^* \to \mathbb{R} \tag{2.26}$$

Let

$$\tau = \langle (s_0, a_0), (s_1, a_1), \dots, (s_{n-1}, a_{n-1}) \rangle \tag{2.27}$$

be a finite trace, and let $\tau_{\leq i}$ denote its prefix containing the first i state-action pairs. The value of τ is defined as

$$V(\tau) = \sum_{i=1}^{|\tau|} \gamma^{i-1} \overline{R}(\tau_{\leq i}) \tag{2.28}$$

Unlike a Markovian policy, whose action depends only on the current state, a policy for an NMRDP may depend on the complete sequence of states observed so far:

$$\overline{\pi} : \mathcal{S}^* \to \mathcal{A} \tag{2.29}$$

Therefore, both the reward and the policy may depend on the interaction history rather than exclusively on the current state.

2.2.2 Linear Temporal Logic on Finite Traces

Linear Temporal Logic (LTL) [12] is a formal language used to describe properties of sequences that evolve over time. Unlike propositional logic, which evaluates formulas with respect to a single state (the *interpretation*), temporal logic allows the specification of relationships between events occurring at different points of an execution.
LTL is interpreted over infinite traces. However, in episodic RL, interactions agent-environment are naturally represented by finite sequences, since an episode terminates after a finite number of steps. Therefore, in this thesis, we mainly consider LTL_f which is a variant of LTL evaluated on finite traces [7].

Let P be a finite set of atomic propositions describing relevant properties of the environment. At each time step, a subset of these propositions is true. A finite trace over P can therefore be represented as

$$\rho = \rho_0, \rho_1, \dots, \rho_{n-1} \tag{2.30}$$

where

$$\rho_i \in 2^P \tag{2.31}$$

denotes the set of atomic propositions that hold at position i.
An LTL_f formula is built from atomic propositions using boolean and temporal operators. Formally,

$$\varphi ::= A \mid (\neg\varphi) \mid (\varphi_1 \wedge \varphi_2) \mid (\circ\varphi) \mid (\varphi_1 \, U \, \varphi_2) \mid (\Diamond\varphi) \mid (\Box\varphi) \tag{2.32}$$

where $A \in P$, $\circ$ is the *next* operator, U is the *until* operator, $\Diamond$ is the *eventually* operator and $\Box$ it the *globally* operator.
The satisfaction of an LTL_f formula is defined with respect to a position of a finite trace. For an atomic proposition p,

$$\rho, i \models p \quad \iff \quad p \in \rho_i \tag{2.33}$$

A finite trace ρ satisfies an LTL$_f$ formula φ, written as

$$\rho \models \varphi \tag{2.34}$$

when the formula is satisfied at the initial position of the trace. Hence, every LTL$_f$ formula defines a set of finite traces that satisfy the corresponding temporal specification.

2.2.3 Deterministic Finite Automata

A Deterministic Finite Automaton (DFA) [2] is a finite-state machine useful to recognize languages defined over a finite alphabet. Formally, a DFA is defined as a tuple

$$\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle \tag{2.35}$$

where Q is a finite set of states, Σ is a finite input alphabet, δ is the transition function

$$\delta : Q \times \Sigma \to Q \tag{2.36}$$

$q_0 \in Q$ is the initial state, and $F \subseteq Q$ is the set of accepting states.
The automaton processes an input word one symbol at a time. Since the transition function is deterministic, for every state $q \in Q$ and input symbol $\sigma \in \Sigma$, there exists a unique successor state

$$q' = \delta(q, \sigma) \tag{2.37}$$

Given a finite word

$$w = \sigma_0 \sigma_1 \dots \sigma_{n-1} \in \Sigma^* \tag{2.38}$$

the execution, or run, of the automaton over w is the sequence of states

$$q_0, q_1, \dots, q_n \tag{2.39}$$

such that

$$q_{i+1} = \delta(q_i, \sigma_i) \qquad 0 \leq i < n \tag{2.40}$$

The word w is accepted by the automaton if the state reached after processing the complete word belongs to the accepting set:

$$q_n \in F \tag{2.41}$$

The set of all finite words accepted by $\mathcal{A}$ defines the language recognized by the automaton, and it is denoted by

$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \mathcal{A} \text{ accepts } w\} \tag{2.42}$$

2.2.4 From LTL$_f$ Specifications to DFA

A LTL$_f$ formula can be represented trough an equivalent DFA. As described in [7], this property is achieved thanks to the fact that every LTL$_f$ formula defines a regular language over finite traces.
Formally, let's consider the LTL$_f$ formula φ defined over a set P of atomic propositions. We can define the DFA $\mathcal{A}_\varphi = \langle Q, 2^P, \delta, q_0, F\rangle$ in such a way that for every finite trace ρ holds:

$$\rho \models \varphi \quad \iff \quad \rho \in L(\mathcal{A}_\varphi) \tag{2.43}$$

Therefore, the LTL$_f$ formula and the corresponding DFA provide two different representations of the same temporal specification. In particular, the former provides a declaratively way to describe a temporal specification, instead the latter provides an operational representation useful for the integration of the formula in an algorithm.

The DFA state summarizes the information contained in the observed trace that is relevant to the satisfaction of the temporal specification. Hence, we can interpret it as a finite-memory representation of the progress made toward the task.
This representation is particularly useful for temporally extended tasks. If we consider different trajectories that lead the environment in the same physical state at a certain time, we will be able to identify the current correct stage of the temporal objective in each trajectory. Hence, the environment state and the automaton state can be considered jointly through an augmented representation

$$\tilde{s}_t = (s_t, q_t), \tag{2.44}$$

where s_t represents the physical configuration of the environment and q_t corresponds to the logical progress with respect to the LTL$_f$ specification. This representation makes the task-relevant history explicitly available to the learning process and it is the basis for the treatment of temporally extended objectives adopted in the framework presented in Chapter 3.

This construction has been formalized in the literature on NMRDP. In particular, in [4] it is shown that LTL$_f$/LDLf non-Markovian rewards can be compiled into an equivalent Markovian MDP by augmenting the environment state with the states of the corresponding automata.

2.3 Reward Design in Reinforcement Learning

The reward design task plays a central role in a RL setting. As depicted in Figure 2.1, the reward received by the agent represents a feedback related on its actions. Therefore, the reward function can be considered as an encoding of which actions are desirable with respect to the task objective required by the problem. Unfortunately, it doesn't exist a general method to the define a reward function. Each reward function must be suitable and informative for the problem under consideration. In a common scenario, the most intuitive reward function corresponds to feeding the agent with a reward only at the end of the task. Although this formulation is trivial, it makes the learning process more difficult. On the other hand, defining a reward function that generates also intermediate feedback could be cumbersome and not straightforward.

In this section, we report some notions about reward functions with a particular focus to sparse reward settings. Then, we describe the Reward Shaping mechanism useful to enrich in some way the reward signal that comes out from the reward function.

2.3.1 Goal-Oriented Tasks and Sparse Rewards

Many RL problems are naturally formulated in terms of achieving a specific objective. In these scenarios, the agent receives a reward signal only when it reaches the final goal, and therefore it is not guided through a reward signal that continuously assesses every intermediate action. Such problems can be modeled as goal-oriented MDPs, in which the task objective is explicitly associated with a goal condition.

Let g denote a goal and let $\mathcal{G}_g \subseteq \mathcal{S}$ be the set of states that satisfy it. A simple sparse formulation assigns a positive reward only when a transition reaches a goal state:

$$R_g(s, a, s') = \begin{cases} r_g, & \text{if } s' \in \mathcal{G}_g, \\ 0, & \text{otherwise} \end{cases} \tag{2.45}$$

where $r_g > 0$ is the reward associated with successful goal achievement.
Depending on the setting, reaching a goal state may also terminate the episode.

A reward function is described as *sparse* when the agent receives informative feedback only on a small fraction of the transitions it experiences. On the other hand, when the agent receives a high frequency reward signal from the environment, we refer to *dense* reward function.

A notable sparse-reward setting is the *goal MDP*: an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ is a *goal MDP* if and only if there exists a set $\mathcal{G} \subseteq \mathcal{S}$ of goal states for which the reward function satisfies

$$R(s, a, s') = \begin{cases} 1, & \text{if } s \notin \mathcal{G} \text{ and } s' \in \mathcal{G}, \\ 0, & \text{otherwise} \end{cases} \tag{2.46}$$

In addition, goal states have zero optimal continuation value:

$$V^*(s) = 0, \qquad \forall s \in \mathcal{G} \tag{2.47}$$

Under these conditions, the agent receives a unit reward only when it enters the goal set from a non-goal state. Once a goal state has been reached, no further return can be accumulated from that state.
Sparse rewards provide a direct representation of the task objective because they reward behavior that actually achieves the final goal. However, they also make learning more difficult [13, 6]. Before encountering a successful trajectory, the agent may receive the same reward for many different actions and states, with no direct

indication of whether its behavior is moving toward or away from the objective. This difficulty is closely related to exploration. Without informative intermediate feedback, the agent must discover successful behavior largely through interaction with the environment. If the goal can be reached only after a long sequence of appropriate decisions, the probability of finding a successful trajectory without the reward signal's guidance may be very low.
Sparse-reward problems will be central into the framework described in Chapter 3.

2.3.2 Reward Shaping and PBRS

To address the consequences of sparse reward settings, in the literature has been introduced a technique called Reward Shaping. The core idea behind reward shaping is to augment the reward signal generated by the reward function of the original MDP with an additional signal [10]. This additional signal provides intermediate feedback to the agent during the learning process.

Formally, given a MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ a shaping function F is useful to define a modified reward function:

$$R'(s, a, s') = R(s, a, s') + F(s, a, s') \tag{2.48}$$

The introduction of F transforms the original MDP M to a modified version $\mathcal{M}' = \langle \mathcal{S}, \mathcal{A}, P, R', \gamma \rangle$. This transformation could modify the desirability of trajectories and the optimal policy induced by the original reward function.

Potential Based Reward Shaping

One of the most adopted solution in the designing of the shaping function F is the Potential Based Reward Shaping (PBRS) [10]. Formally,

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s) \tag{2.49}$$

From the definition, it is immediate to see that the function $F(s', a, s)$ does not depend on the action a and therefore it could be defined simply as $F(s, s')$. Moreover, there is the introduction of $\Phi(s)$ which is the *Potential Function*. Specifically,

$$\Phi : S \to \mathbb{R} \tag{2.50}$$

assigns a scalar potential value to each state. Despite the difference in meaning between Φ and V-function, they share a structural similarity in their definition. In fact, the V-function is a practical and common choice when the reward shaping technique is adopted.

Policy Invariance of PBRS

The introduction of the shaping term according on PBRS formulation does not alter the optimal policies of the original MDP, if we assume the following facts:

- the discount factor $\gamma \in [0, 1)$;
- the potential function Φ is bounded.

This result relies on the fact that the shaping contribution accumulated along a trajectory assumes a telescoping structure. Considering a trajectory

$$s_0, s_1, \ldots, s_T$$

the discounted cumulative shaping reward is

$$\sum_{t=0}^{T-1} \gamma^t \left(\gamma\Phi(s_{t+1}) - \Phi(s_t)\right) = -\Phi(s_0) + \gamma^T \Phi(s_T) \tag{2.51}$$

Under the assumptions:

$$\lim_{T\to\infty} \gamma^T \Phi(s_T) = 0$$

and therefore

$$\sum_{t=0}^{\infty} \gamma^t F(s_t, s_{t+1}) = -\Phi(s_0) \tag{2.52}$$

For a fixed initial state s_0, the shaped return differs from the original one only for $-\Phi(s_0)$. Hence, the relative ordering of policies is preserved, and any optimal policy in the original MDP remains optimal in the shaped MDP [10].
When we are in episodic finite-horizon settings, additional care is required because the terminal term $\gamma^T \Phi(s_T) \neq 0$. A common sufficient condition is to assign zero potential to terminal states:

$$\Phi(s) = 0, \qquad \forall s \in \mathcal{S}_{\text{terminal}} \tag{2.53}$$

so that, the cumulative shaping contribution again reduces to $-\Phi(s_0)$. Under this condition, the shaping transformation preserves the original task objective also in the episodic case.

Chapter 3

Multi-Layer Abstraction Framework

The foundations of this thesis lie in the multi-level framework introduced in [6]. The framework exploits abstract representations of the original decision process in order to tackle the difficulties associated with a sparse-reward learning problem.

In this chapter, we firstly analyze the main components of the reference framework including the abstraction hierarchy and the learning architecture adopted. Then, we move on presenting the resulting end-to-end algorithm. In the end, we describe its implementation and the integration of non-Markovian goal specifications trough LTL_f.

3.1 State Abstraction and Abstract Guidance

The core idea behind the framework corresponds to the linear hierarchy of MDPs built over the ground domain. In particular, each level of the stack can be seen as a simplification of the one immediately below. The purpose of this hierarchical organization is to exploit simpler representation of the original decision problem in order to derive additional information that can guide the learning process in a more efficient way.
Let

$$\mathcal{M}_0, \mathcal{M}_1, \ldots, \mathcal{M}_L$$

denote the sequence of MDPs forming the hierarchy, where $\mathcal{M}_0$ represents the ground MDP. Therefore, the hierarchy can be conceptually represented as

$$\mathcal{M}_L \rightarrow \mathcal{M}_{L-1} \rightarrow \cdots \rightarrow \mathcal{M}_0$$

where the information flows according to the arrows.

3.1.1 State Abstraction mechanism

Each component of the hierarchy is a complete MDP $\mathcal{M}_i = \langle \mathcal{S}_i, \mathcal{A}_i, P_i, R_i, \gamma_i \rangle$. Given two consecutive levels of the hierarchy, $\mathcal{M}_i$ and $\mathcal{M}_{i+1}$, the latter represents a simplified model of the former. They are connected trough an abstraction mapping, which is a function that maps the states of $\mathcal{M}_i$ into the states of $\mathcal{M}_{i+1}$. Formally the mapping function is

$$\varphi_i : \mathcal{S}_i \to \mathcal{S}_{i+1} \tag{3.1}$$

and the pair $\langle \mathcal{M}_{i+1}, \varphi_i \rangle$ is the abstraction built over $\mathcal{M}_i$. Unlike other HRL frameworks, this one does not impose any constraint about the mapping between $\mathcal{A}_i$ and $\mathcal{A}_{i+1}$. This feature leaves freedom and flexibility to the designer during the definition of the abstractions. Therefore, each MDP can be equipped with an appropriate set of actions suitable for each level of the problem description. The only assumptions made in [6] are:

- $\mathcal{M}_0$ is a Goal MDP;
- if $\mathcal{M}_i$ is a goal MDP with goal states $\mathcal{G}_i$ and $\langle \mathcal{M}_{i+1}, \varphi_i \rangle$ is the abstraction built over it, then $\mathcal{M}_{i+1}$ is a goal MDP such that:

$$\mathcal{G}_{i+1} = \bigcup_{s \in \mathcal{G}_i} \varphi_i(s) \tag{3.2}$$

3.1.2 Guidance from Abstract Models

Once the abstraction hierarchy has been defined, the framework exploits the higher-level models to derive guidance for the learning process at lower levels. The main idea is to solve an abstract MDP and use the resulting information in some way to speed-up the learning phase at the lower level.

Let $\mathcal{M}_{i+1}$ an MDP of the hierarchy. We can solve it using planning or learning techniques. In particular, we can adopt the first solutions when the MDP is fully-known, including the transition function and the reward function. This is not always true, especially when we are considering a rich and complex abstraction over the real problem. For instance, an abstraction that includes in its dynamics physics relations and quantities.
In the framework, the information extracted at level $(i + 1)$ corresponds to V_{i+1}. Then, this term is plugged inside the PBRS formulation described in Section 2.3.2:

$$F_i(s, a, s') = \gamma V_{i+1}(s') - V_{i+1}(s) \tag{3.3}$$

$F_i(s, a, s')$ corresponds to the shaping term exploited to enrich the reward function at level i. As a consequence of this choice, transitions in $\mathcal{M}_i$ that leads towards states mapped to abstract ones with higher potential values receive an additional positive feedback in the reward signal. On the other hand, transitions towards states associated with lower potential in the abstraction can receive a negative shaping contribution.

3.2 Reward Shaping and Learning Architecture

Although the framework adopts a potential-based shaping formulation, its shaping mechanism differs from the standard return-invariant setting.

Episodic tasks are common settings to be faced in RL. In this situations, preserving the return-invariance property of PBRS requires a particular treatment of terminal states, as discussed in Section 2.3.2. While this property guarantees that the shaped and original MDPs share the same optimal policies, it may also limit the ability of the shaping signal to provide persistent exploration guidance.

3.2.1 Non Return-Invariant Reward Shaping

The return-invariance property of PBRS can be preserved by assigning a null potential to terminal states. Under this condition, for a trajectory $s_0, s_1, \ldots, s_T$, the cumulative discounted shaping contribution is

$$G = -V(s_0) + \gamma^T V(s_T) \tag{3.4}$$

and since $V(s_T) = 0$, the previous expression reduces to $-V(s_0)$.

It is immediate to notice that, for trajectories originating from the same initial state (s_0 is the same) the cumulative shaping reward is independent of the intermediate states visited. Hence, in a sparse reward problem like a goal MDP, all finite trajectories that terminate without reaching the objective will receive the same shaped return contribution, regardless of whether they end close to or far from the desired goal.

To overcome this limitation, the authors of [6] introduce a modified version of PBRS called *non return-invariant Reward Shaping*. In practice, the constraint about the final state value is relaxed.
Now, two unsuccessful trajectories ρ_1 and ρ_2, originating from the same state, can satisfy

$$G(\rho_1) > G(\rho_2)$$

retaining useful information about the quality of the trajectories, with respect to the distance between the final state and the final goal. In other words, adopting this version, the shaping mechanism provides a persistent bias toward states associated with higher abstract values.

3.2.2 Biased and Unbiased Learning

The non return-invariant shaping mechanism loses the guarantee that the optimal policy of the shaped MDP corresponds to the optimal one in the original MDP.
From this consideration, it's now important to consider two MDPs at each level in the hierarchy. Precisely, we have:

- $\mathcal{M}^u = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ named *unbiased* MDP. The reward function R is not influenced by the shaping signal;
- $\mathcal{M}^b = \langle \mathcal{S}, \mathcal{A}, P, R', \gamma \rangle$ named *biased* MDP. Here, the reward function R' involves also the shaping signal according to the non return-invariant shaping formulation.

At this point, the task is to learn an optimal policy in $\mathcal{M}^u$ exploiting the information coming from the learning process performed on $\mathcal{M}^b$. Hence, from now on, we have two learning procedures over the two different MDPs.

The learning procedures are coupled together in such a way we can exploit the advantage coming from the shaping signal without losing the guarantee of reaching the optimal policy in the unbiased MDP. In particular, the biased learner is responsible for interacting with the environment and therefore it establishes the behavior policy used during data collection. The key idea supporting this architecture is to exploit the trajectories generated by the biased learner in $\mathcal{M}^b$ to update also the unbiased one.
Formally, for each transition generated during the interaction with the environment

$$(s_t, a_t, r_{t+1}, s_{t+1})$$

both learning processes are updated using the same experience, but with different reward signals. The biased learner uses

$$r'_{t+1} = r_{t+1} + F(s_t, a, s_{t+1})$$

while the unbiased learner uses only the original reward r_{t+1}. In this way, the unbiased learner is able to see trajectories toward the final goal allowing it to learn the optimal desired policy.
It is worth to mention that this architecture requires the adoption of an off-policy algorithm to implement the unbiased learner. This concept is obvious considering that transitions comes out from a different algorithm (the biased learner one).

3.3 End-to-End Framework

The overall procedure described in the previous sections is summarized in the following algorithm extracted and adapted from [6].

Algorithm 1: Full learning architecture

Input: An off-policy learning algorithm $\mathcal{L}$
Input: The hierarchy $\mathcal{M}_0, \ldots, \mathcal{M}_L$ and the abstraction mappings $\varphi_0, \ldots, \varphi_{L-1}$
Output: An estimate $\widehat{\pi}_0^*$ of the optimal policy for the ground MDP

$\widehat{V}_{L+1}^*(s) \leftarrow 0$;
$\varphi_L(s) \leftarrow s$;
for $i \leftarrow L, L-1, \ldots, 0$ **do**
  $F_i \leftarrow \text{Shaping}\left(\gamma_i, \varphi_i, \widehat{V}_{i+1}^*\right)$;
  $\text{Learner}_i^u \leftarrow \mathcal{L}(\mathcal{M}_i)$;
  $\text{Learner}_i^b \leftarrow \mathcal{L}(\mathcal{M}_i)$;
  while $\neg\,\text{Learner}_i^b.\text{Stop}()$ **do**
    $s \leftarrow \mathcal{M}_i.\text{State}()$;
    $a \leftarrow \text{Learner}_i^b.\text{Action}(s)$;
    $(r, s') \leftarrow \mathcal{M}_i.\text{Act}(a)$;
    $r^b \leftarrow r + F_i(s, a, s')$;
    $\text{Learner}_i^b.\text{Update}(s, a, r^b, s')$;
    $\text{Learner}_i^u.\text{Update}(s, a, r, s')$;
  $\widehat{\pi}_i^* \leftarrow \text{Learner}_i^u.\text{Output}()$;
  $\widehat{V}_i^* \leftarrow \text{ComputeValue}(\widehat{\pi}_i^*, \mathcal{M}_i)$;

The algorithm highlights two important features of the learning architecture:

1. after completing the learning process at level i, the value function used to construct the shaping signal for the level immediately below is derived from the unbiased learner. In particular, the policy produced by the unbiased learner is used to compute the corresponding value function V_i^*, which then serves as the source of guidance for level $i-1$;

2. the final result of the procedure is the policy $\widehat{\pi}_0^*$ learned by the unbiased learner at the ground level.

Hence, it is clear that the biased policies are used only as intermediate mechanisms to improve the exploration process for the unbiased learner.

3.4 Implementation and Design Choices

The framework described in the previous sections represents the foundation of this thesis. Starting from the architecture and the algorithm introduced in [6], a full implementation of the multi-level framework has been developed with the aim to experimentally evaluate its effectiveness against temporally extended non-Markovian task specifications in a complex domain.
Until now, the original framework has been evaluated on relative simple tasks and domains. Moreover, when a temporally extended behavior was required, there was a need to manually define the corresponding DFA for the task.
In this thesis, we address this limitation introducing explicitly the support for task specifications expressed in LTL_f. Furthermore, the whole training pipeline has been fully-automated in such a way to be transparent and *ready-to-use* for the user.

3.4.1 Framework Architecture

The implementation developed in this thesis follows the multi-level architecture described in the previous sections, while extending it with an explicit representation of temporally extended tasks. The temporal objective is specified trough an LTL_f formula, which is automatically translated into a DFA taking advantage of the *ltlf2dfa* Python library. The state q of the automaton is shared across each level in the hierarchy, so that all abstraction levels are synchronized on the same temporal objective over different representations of the environment.

The hierarchy is processed from the highest abstraction level toward the ground MDP. The highest-level MDP is solved without receiving shaping guidance. When its transition and reward models are explicitly available and the state space is sufficiently small, we can also adopt a Value Iteration algorithm to solve the abstract task. Once a level has been solved, the resulting V-function is propagated to the level immediately below and used to construct its shaping signal according to the mechanism described in Algorithm 1. This procedure is repeated throughout the hierarchy until the ground level is reached.

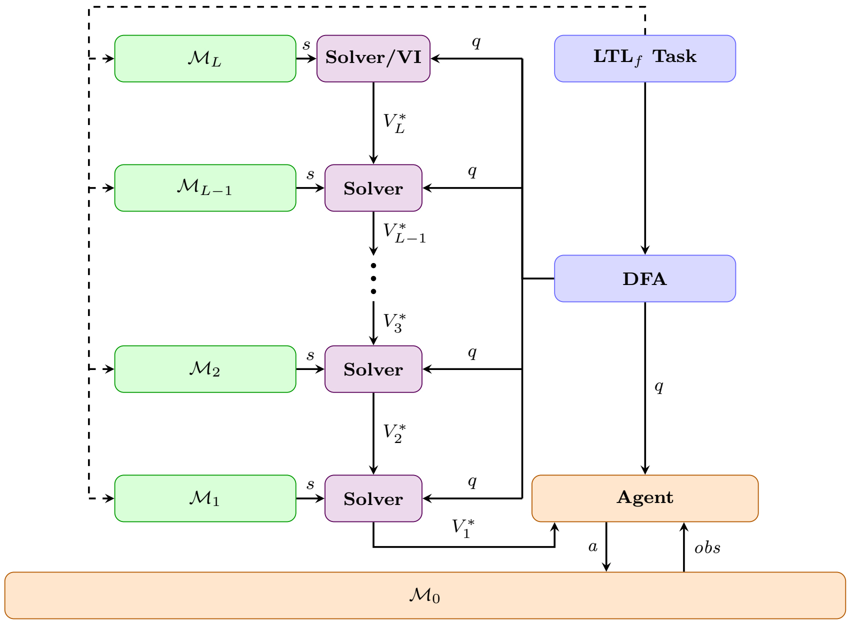

Figure 3.1. Full framework architecture.

At each level, the V-function propagated to the level immediately below is derived from the solution produced by the unbiased learner. The framework supports two different ways to extract the unbiased V-function.
The first possibility consists to extract the greedy policy induced by the unbiased Q-function following

$$\pi^u(s) = \arg\max_{a \in \mathcal{A}} Q^u(s, a) \tag{3.5}$$

The, V^{π^u} is computed trough iterative Policy Evaluation, as introduced in Section 2.1.3. When this solution is adopted, the numerical magnitudes and errors of Q^u are not propagated directly.
Alternatively, the V-function can be obtained by estimating it following

$$\hat{V}^u(s) = \max_{a \in \mathcal{A}} Q^u(s, a) \tag{3.6}$$

Therefore, the magnitudes and estimation errors are propagated trough the shaping mechanism. A comparative study of the two solution will be presented in Chapter 4.

3.4.2 Temporal Task Specification

The implementation supports two different classes of temporally extended tasks:

- episodic tasks: they are expressed trough a LTL$_f$ formula;
- cyclic tasks: they are expressed trough a dedicated automaton whose transitions encode the phases of the recurring task.

Episodic tasks

For episodic tasks, reaching an accepting state of the DFA corresponds to the completion of the temporal objective. The associated goal reward is generated and the current interaction episode terminates. At the beginning of a new episode, both the environment and the automaton state are reinitialized.

Cyclic tasks

For cyclic tasks, reaching the end of one temporal sequence does not terminate the overall objective. Instead, the task progress is redirected to the initial phase of the cycle, allowing the same temporal behavior to be executed again during the same episode. Hence, a LTL_f is not suitable to express this type of required interaction. However, the corresponding automaton is designed to contain transitions that explicitly represent this recurrence.

3.4.3 Configuration Interface and Automated Pipeline

The complete framework is configurable through two external JSON files, one describing the abstraction hierarchy and one specifying the temporal task. This design separates the experimental configuration from the implementation of the learning procedure, allowing different tasks and abstraction settings to be tested without modifying the underlying source code.

The abstraction hierarchy is defined through an `abstraction.json` file. This configuration specifies the number of abstraction levels and the parameters required to instantiate and solve the corresponding abstract MDPs. A possible configuration is the following one:

```json
{
    "levels": [
        {
            "name": "level1", "grid_w": 360, "grid_h": 360,
            "learning": {
                "episodes": 1000000,
                "max_steps": 600,
                "alpha": 0.1,
                "epsilon_start": 1.0,
                "epsilon_min": 0.05,
                "epsilon_decay": 0.999995,
                "gamma_shaping": 1.0
            }
        },
        {
            "name": "level2", "grid_w": 180, "grid_h": 180,
            "algorithm": "value_iteration"
        }
    ]
}
```

Listing 3.1. Example of an abstraction hierarchy configuration.

The temporal task is specified through a separate configuration file `trajectory.json`. In the episodic case, the configuration contains the LTL$_f$ formula together with the definitions required to evaluate the atomic propositions appearing in it.
In the cyclic case, the recurring temporal structure is explicitly encoded and from that specification the automaton is automatically built.

A possible episodic task configuration is the following one:

```
{
    "formula": "F(wp1 & X(F(low & g1)))",
    "goal_reward": 10000,
    "regions": {
        "wp1": {
            "center": [-0.5, 1.0],
            "radius": 0.1
        },
        "g1": {
            "center": [0.5, 0.5],
            "radius": 0.1
        }
    },
    "predicates": {
        "low": {
            "type": "half_plane",
            "axis": "y",
            "operator": "<",
            "threshold": 0.7
        }
    }
}
```

Listing 3.2. Example of an LTL$_f$ task configuration.

A possible cyclic task configuration is the following one:

```
{
    "task_type": "cyclic_waypoints",
    "goal_reward": 10000,
    "waypoint_cycle": ["g1", "g2", "g3", "g4"],
    "regions": {
        "g1": {"center": [-0.75, 1.0625], "radius": 0.12},
        "g2": {"center": [0.583333, 1.0625], "radius": 0.12},
        "g3": {"center": [0.583333, 0.4375], "radius": 0.12},
        "g4": {"center": [-0.75, 0.4375], "radius": 0.12}
    }
}
```

Listing 3.3. Example of a cyclic task configuration.

The predicates shown in these examples are related to the specific ground domain used for the experiments. For our use-case, the definition of the relevant predicates will be described in Chapter 4.

Chapter 4

Experimental Evaluation

In this chapter, we present the experimental evaluation of the implemented framework applied to a complex domain with temporally extended non-Markovian task specifications.
The experiments have been designed to investigate the effectiveness and applicability of the learning architecture under different task and abstraction configurations.
Throughout this chapter, we will firstly evaluate the framework on episodic tasks of increasing temporal complexity, then we will study the effect of introducing multiple abstraction levels, and finally we will consider its application on cyclic temporal objectives.

4.1 Evaluation Goals and Research Questions

The target of this experimental analysis is to identify possible limitations of the framework and to better understand its potential when increasingly complex tasks must be fulfilled. The experimental evaluation pipeline is organized around four key research questions, each one investigating a different aspect of the framework.

Our research question are:

- **RQ1** To what extent can the theoretical framework be directly applied to a complex domain without requiring additional adaptations?
- **RQ2** How does the multi-level framework perform on episodic non-Markovian tasks with a growing temporal complexity?
- **RQ3** How does the introduction of multiple abstraction levels affect the guidance signal and the resulting learning process?
- **RQ4** Can the framework be applied to learn cyclic temporal behaviors that are not limited to purely episodic task structures?

These research questions guide the experimental analysis presented in the following sections, covering the practical limitations of the learning architecture, the effect of temporal-task complexity, the role of multi-level abstraction, and the extension to cyclic task structures.

4.2 Experimental Setting

4.2.1 Environment and Abstractions

To carry out the experimental evaluation we have chosen the *Gymnasium Lunar Lander* environment. Over it, the framework supports the possibility to equip different abstract grids used to generate the shaping guidance.

Gymnasium Lunar Lander

This environment is a classic rocket trajectory optimization problem. The agent is a lander with the goal to correctly lands on the ground in between of a specified region. The agent is equipped with three engine that can be turned on/off during the interaction with the environment.

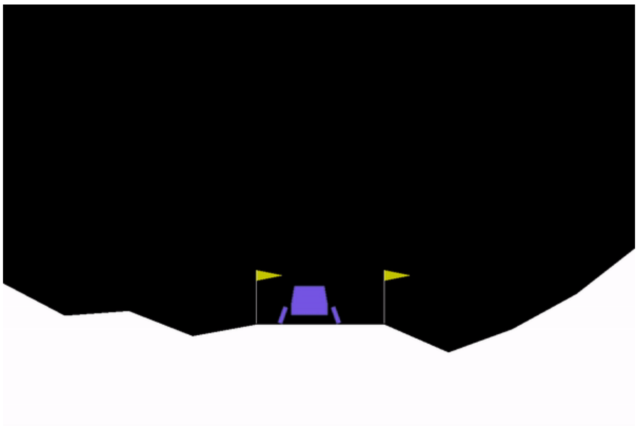

Figure 4.1. The Gymnasium Lunar Lander environment.

The state space is represented by an eight-dimensional observation vector where:

- x and y denotes the horizontal and vertical position of the lander;
- v_x and v_y denotes the linear velocities of the lander;
- θ denotes the lander orientation;
- ω denotes the angular velocity of the lander;
- c_l and c_r denotes whether the left and right legs are in contact with the ground.

The action space is discrete and provides four different actions:

- 0 do nothing;
- 1 fire left orientation engine;
- 2 fire main engine;
- 3 fire right orientation engine.

The original *Lunar Lander* environment is characterized by a complex and dense reward function. For the purposes of this thesis, the original reward function is not used. Instead, the reward structure is replaced by a sparse goal-based formulation, so that the ground environment can be treated as a Goal MDP with respect to the temporal objective considered in each experiment.
For this reason, the details of the original Lunar Lander reward function are intentionally omitted. Moreover, the original landing objective is not considered: the goals adopted throughout the experimental evaluation are defined by the temporal task specifications introduced in the following sections.

Abstractions Grids

The abstract environments supported by the framework are represented trough 2D discrete grids. Each grid provides a simplified representation of the ground Lunar Lander state space considering only the positional components of the state space.
In particular, the abstraction is defined over the x and y components. The other physical quantities are intentionally discarded. The purpose of the abstraction is not to reproduce the complete dynamics of Lunar Lander, but to provide a simpler state representation from which task-related guidance can be derived.

Formally, an abstraction grid is a MDP $\mathcal{M}_{\text{grid}} = \langle \mathcal{S}_{\text{grid}}, \mathcal{A}_{\text{grid}}, P_{\text{grid}}, R_{\text{grid}}, \gamma \rangle$ where:

- $\mathcal{S}_{\text{grid}} = \{0, \dots, W-1\} \times \{0, \dots, H-1\}$;
- $\mathcal{A}_{\text{grid}} = \{\uparrow, \downarrow, \leftarrow, \rightarrow \nwarrow, \nearrow, \swarrow, \searrow, \mathit{Done}\}$;
- $P_{\text{grid}} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \{0, 1\}$ is the deterministic transition function, defined as

  $$P_{\text{grid}}(s' \mid s, a) = \begin{cases} 1, & \text{if } s' = T(s, a), \\ 0, & \text{otherwise} \end{cases}$$

  where T denotes the successor state obtained by applying action a to state s;
- $R_{\text{grid}} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the classical reward function used in a Goal MDP setting, as described in Section 2.3.1;
- $\gamma \in [0, 1]$ is the discount factor.

State Abstraction Mapping

The ground MDP and the abstraction built over it are linked together trough several mapping function φ_i, as described in Section 3.1.1.

In particular, two different mapping function types are required to correctly implement the hierarchy. The first mapping connects the continuous ground environment to the first abstract grid. It corresponds to a classical projection operation between a point in the continuous space to a cell in the abstract grid. The second mapping is instead defined between two consecutive abstract grid levels. Now, since both source and target states corresponds to grid cells, the mapping is performed by projecting the center of the source cell to the target grid.

For the ground-to-grid mapping, let the continuous spatial domain be bounded by $x \in [x_{\min}, x_{\max}]$ and $y \in [y_{\min}, y_{\max}]$. Let the first abstract grid have width W and height H. Given a continuous position (x, y), the corresponding abstract cell (i, j) is computed as

$$i = clip\left(\left\lfloor \frac{x - x_{\min}}{x_{\max} - x_{\min}} W \right\rfloor, W - 1\right) \tag{4.1}$$

$$j = clip\left(\left\lfloor \frac{y - y_{\min}}{y_{\max} - y_{\min}} H \right\rfloor, H - 1\right) \tag{4.2}$$

In the experiments $x \in [-1, 1]$ and $y \in [0, 1.5]$. These intervals do not correspond to the real environment ones, but they correctly describe the visible area.

For the mapping between two consecutive abstraction grids, let the source grid have size $W_s \times H_s$ and the target grid size $W_t \times H_t$. Given a source cell (x, y), the center of the cell is projected onto the target grid according to

$$x' = clip\left(\left\lfloor \frac{x + 0.5}{W_s} W_t \right\rfloor, W_t - 1\right) \tag{4.3}$$

$$y' = clip\left(\left\lfloor \frac{y + 0.5}{H_s} H_t \right\rfloor, H_t - 1\right) \tag{4.4}$$

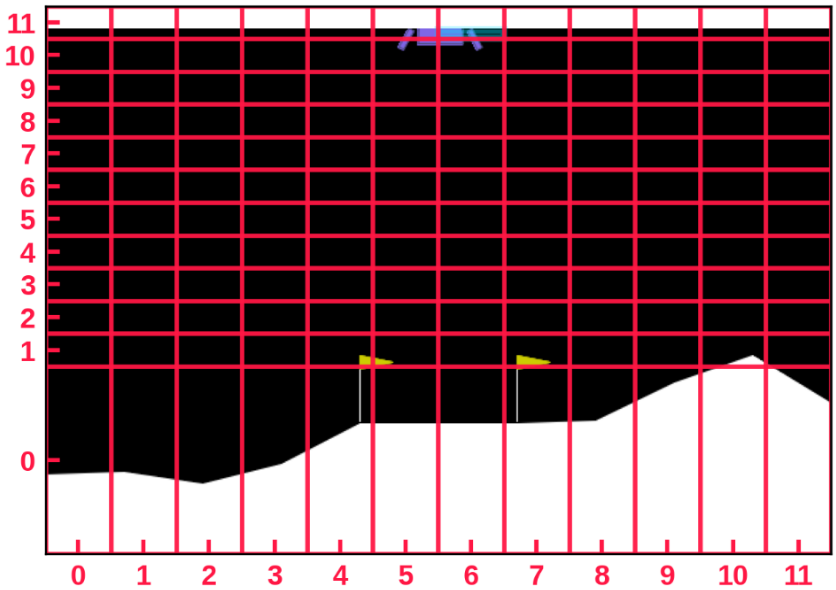

Figure 4.2. Example of a grid abstraction 12x12 over the Lunar Lander environment.

4.2.2 Temporal Tasks and Predicates

The temporal tasks considered in the experiments are defined over a set of spatial predicates, which are evaluated on the spatial coordinates (x, y) of the Lunar Lander environment. The set of atomic propositions used to define a task is named AP.

Supported Predicates

The frameworks supports three different predicate types:

- *circular region*: a circular predicate is defined specifying a center (c_x, c_y) and a radius r. The predicate is satisfied when

$$(x - c_x)^2 + (y - c_y)^2 \leq r^2$$

- *rectangular region*: a rectangular region is defined specifying the bounds x_{min}, x_{max}, y_{min}, and y_{max}. The predicate is satisfied when

$$x_{\text{min}} \leq x \leq x_{\text{max}} \quad \wedge \quad y_{\text{min}} \leq y \leq y_{\text{max}}$$

- *half planes*: an half-plane corresponds to a spatial condition with respect to a certain threshold along a coordinate axis. The framework supports half-planes along both axis.

Predicate mapping on Abstraction Grids

The predicates involved in the task specification are defined over the continuous ground state space. This fact guarantees that the task specification is completely independent from whatever abstraction grids have been adopted to solve the problem. In order to evaluate the same task specification over different levels in the hierarchy, each predicate needs to be rasterized onto the corresponding grid.

Formally, let R_p denote the continuous region associated with predicate p and let C_{ij} denote the rectangular area represented by the grid cell (i, j). The predicate is assigned to the cell whenever:

$$C_{ij} \cap R_p \neq \emptyset \tag{4.5}$$

This criterion corresponds to an over-approximation of the original continuous region, since a predicate is considered true in the whole cell even when only part of the cell intersects the corresponding region. This is done to test the reliability of the framework when the abstractions are not fully precise. Moreover, the rasterization task is performed for each level in the hierarchy starting from the continuous domain. This choice has been intentionally adopted to avoid the over-approximation error propagation between the abstractions levels.

At level k, the result of the rasterization is encoded through a labeling function

$$L_k : \mathcal{S}_k \to 2^{AP} \tag{4.6}$$

defined as

$$L_k(i, j) = \left\{ p \in AP \mid C_{ij}^{(k)} \cap R_p \neq \emptyset \right\} \tag{4.7}$$

Augmented state representation

The spatial state alone is not sufficient to represent the progress of a Non-Markovian temporal task. For this reason, at each level in the hierarchy, we need to consider the augmented version of the state space, as described in Section 2.2.4.

After each transition, the predicates that hold in the successor state are evaluated through the corresponding labeling function. Let $\sigma \subseteq AP$ denote the set of atomic propositions that are true after the transition. The automaton state is then updated according to

$$q' = \delta(q, \sigma) \tag{4.8}$$

Therefore, a transition at abstraction level i can be represented as

$$(x_i, y_i, q) \xrightarrow{a} (x'_i, y'_i, q') \tag{4.9}$$

with

$$q' = \delta\left(q, L_i(x'_i, y'_i)\right) \tag{4.10}$$

At this point, we need to consider also an extension to the mapping function φ_i. We know that the logical task progress must remain aligned across all levels. Hence, we can extend the mapping function as follows:

$$\tilde{\varphi}_i(x_i, y_i, q) = (\varphi(x_i, y_i), q) \tag{4.11}$$

Furthermore, each time a new episode is starting we label the initial state s_0 before the first action is executed. In this way, the DFA can account for predicates that are already satisfied at the beginning of the interaction.

4.2.3 Learning and Evaluation Setup

The experiments presented throughout this chapter are carried out trying to use a common architecture, while the specific hyperparameters may vary depending on the considered experimental scenario.

Learning Algorithms

The framework is equipped with a set of learning algorithms: DDQN, Dueling-DDQN and Tabular Q-Learning. All of them have been described in Section 2.1.4.

The ground MDP is characterized by a continuous observation space and therefore requires a neural approach to estimate the action-value function. For this reason the choice was DDQN.
Instead, at the abstract levels, the state spaces are discrete and significantly smaller. For this reason, Tabular Q-Learning has been adopted.
The two learning procedures are configured independently, since the ground and abstract MDPs differ substantially in terms of state-space representation and complexity. The main hyperparameters and configurations adopted for DDQN and Tabular Q-Learning are reported in the following tables.

Parameter	Value
Network architecture	MLP $(128, 128)$
Hidden activation	ReLU
Input dimension	$8 + \|Q\|$
Output dimension	4
Optimizer	Adam
Learning rate	10^{-3}
Loss function	MSE
Batch size	64
Replay-buffer capacity	300 000 transitions
Replay sampling	Uniform
Discount factor γ	0.99
Target-network update	Polyak averaging
Polyak coefficient τ	0.005

Table 4.1. DDQN configuration

Parameter	Value
Learning rate α	0.1
Discount factor γ	0.99
Q-value initialization	0
State initialization	Uniform random restart
Update frequency	One update per transition

Table 4.2. Q-learning configuration.

During the learning episodes, in both cases, it has been adopted the classical ϵ-greedy strategy. Its configuration may depend on the specific experiment. Furthermore, in all the experiments, the reward assigned when a task is completed is set to 10000. From now on, this reward is referred as *task reward* or *environment reward*.

Evaluation Procedure and Metrics

During the learning procedures, at each levels, several training and evaluation metrics have been monitored. In particular, the performance of the agent is monitored during training through periodic greedy evaluations, in which exploration is disabled. At the end of each learning procedures the results are two different policies:

- **best policy** which is selected among all the checkpoints evaluated during the training task;
- **last policy** which corresponds to the policy resulting after executing the whole training task.

The main metrics adopted throughout the experimental evaluation are:

- **task reward**: cumulative reward associated with the temporal task during an episode. It does not involve the shaping signal;

- **shaping reward**: cumulative contribution generated by the abstraction-based shaping signal.
- **success rate**: fraction of evaluation episodes in which the temporal objective is successfully completed. For cyclic task this means that at least one cycle has been completed during the interaction.
- **episode length**: number of interaction steps performed before termination;
- **DFA progression**: statistics related to the automaton states and transitions visited during the interaction.

When multiple seeds are considered, results are aggregated across runs by reporting the mean value together with a variability band representing the standard deviation. Furthermore, to select the best policy among the periodically evaluated checkpoints, a hierarchical criterion is adopted:

1. highest success rate;
2. highest mean task reward. For cyclic tasks, the latter corresponds to the highest mean number of completed cycles;
3. lowest mean episode length;
4. earliest checkpoint.

Setup

4.3 RQ1: Direct Applicability of the Framework

This section investigates whether the framework is directly applicable to the considered ground domain. These experiments are organized to analyze two important aspects of the implemented learning architecture. The first aspect studied is the interaction between the biased and unbiased learners. Instead, the second one is aimed to study the real impact of the shaping guidance on the learning process.

4.3.1 Biased-Unbiased Interaction

We recall from Section 3.2.2 that the architecture employs two learners with different objectives. The biased learner interacts with the environment using the shaped reward (task reward augmented of the shaping signal), while the unbiased learner is updated on the same experience generated by the biased one, but considering only the original task reward.

To isolate the effect of this coupling, the settings considered are:

- *1-phase trajectory* with unbiased standalone learner;
- *1-phase trajectory* with coupled learner architecture;
- *2-phase trajectory* with unbiased standalone learner;
- *2-phase trajectory* with coupled learner architecture.

1-Phase Trajectory

This task corresponds to a trajectory composed by only one phase: reaching a specific region of the environment. The corresponding LTL$_f$ formula is:

$$\varphi = \Diamond g_1$$

where g_1 is the predicate associated to the target region.

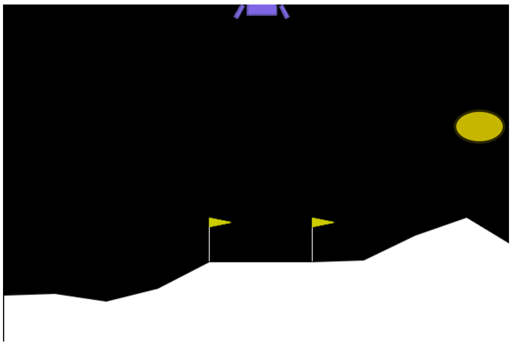

Figure 4.3. 1-phase task.

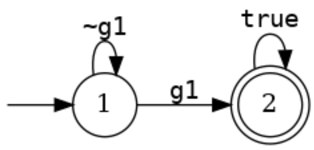

Figure 4.4. DFA for the 1-phase task.

2-Phase Trajectory

This task corresponds to a trajectory composed by two phases: reaching a first waypoint and then the target region of the environment. The corresponding LTL$_f$ formula is:

$$\varphi = \Diamond (\mathrm{wp}_1 \wedge \circ (\Diamond g_1))$$

where wp_1 and g_1 are the predicates associated to the waypoint and the target region, respectively.

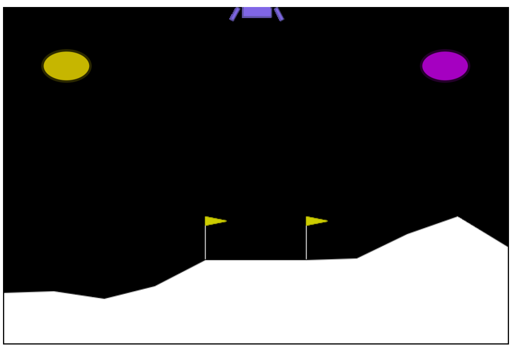

Figure 4.5. 2-phase task. In yellow wp_1 and in purple g_1.

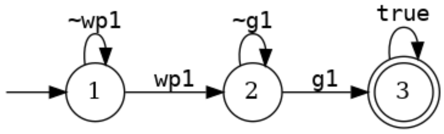

Figure 4.6. DFA for the 2-phase task.

Hyperparameters

Here, are reported the common hyperparameters adopted for these experiments.

Parameter	Value
Episodes	10000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	5
Grid Dimension	12×12
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes	1000

Table 4.3. Hyperparameters adopted in the experiments.

The grid resolution affects only the experiments with coupled learner.

Results and Interpretations

We evaluate the outcome of these experiments considering the evaluation performances of the various learners during the training procedure.

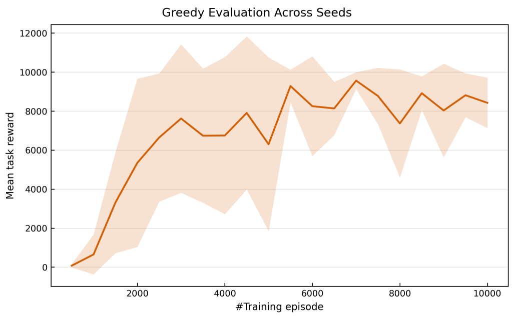

Figure 4.7. 1-phase task. Periodically greedy evaluation of the standalone unbiased policy.

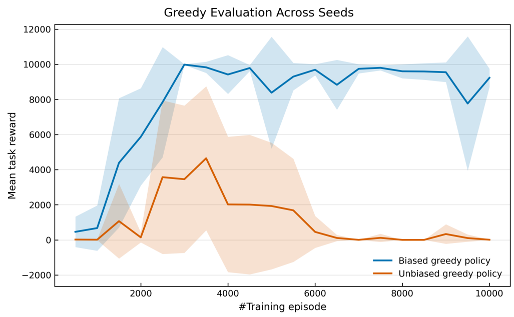

Figure 4.8. 1-phase task. Periodically greedy evaluation of the biased and unbiased policies.

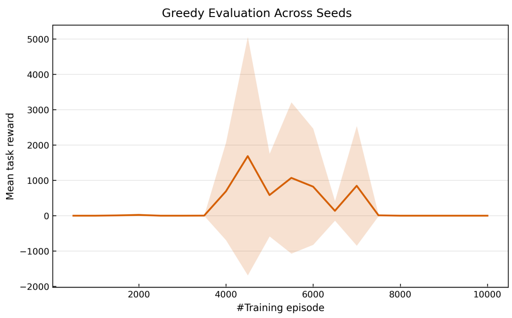

Figure 4.9. 2-phase task. Periodically greedy evaluation of the standalone unbiased policy.

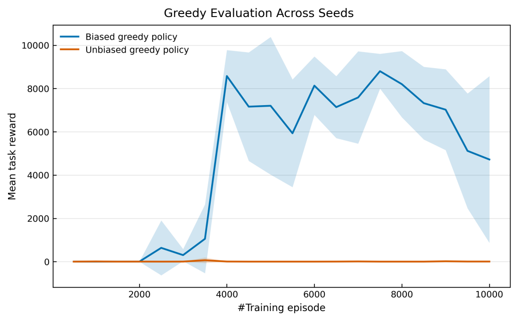

Figure 4.10. 2-phase task. Periodically greedy evaluation of the biased and unbiased policies.

The results obtained provide a first indication of a practical limitation in the coupled learning architecture. Specifically, in the 1-phase setting, the standalone unbiased learner was able to learn a policy that fulfills the task. On the other hand, in the coupled architecture, the unbiased learner does not maintain a stable and effective policy throughout training, even though the biased learner successfully solves the task.

More precisely, during the early stages of training, the unbiased learner was able to discover an useful policy but it didn't manage to preserve its until the end. Since the main difference with respect to the standalone case lies in the source of the collected experience, this is a strong suggestion that the biased learner strongly influences the data distribution seen by the unbiased one and the introduced effect is catastrophic.

Moving on the 2-phase setting, we immediately notice that the task required is too difficult to be learned by the standalone unbiased learner. The only learner that was able to build a strong and useful policy is the biased one. Therefore, we can deduce that the shaping information extracted from the 12×12 grid was crucial to solve the task.

4.3.2 Shaping Guidance

We recall the shaping formulation used in the framework, from Section 3.1.2:

$$F(s, a, s') = \gamma V(s') - V(s)$$

From now on, we will refer to the γ factor involved in the formula as γ_s.

To analyze the shaping guidance quality, we focus on the impact of γ_s. In particular, the tasks considered are:

- *2-phase trajectory* with $\gamma_s = 0.99$;
- *2-phase trajectory* with $\gamma_s = 1$.

The *2-phase trajectory* considered is the one already studied in the previous section.

Hyperparameters

Here, are reported the common hyperparameters adopted for these experiments.

Parameter	**Value**
Episodes	10000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	5
Grid Dimension	12×12
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes	1000

Table 4.4. Hyperparameters adopted in the experiments.

Results and Interpretations

The analysis has been carried out on the metrics related to the biased learner's performance. We already know that the unbiased learner is not able to learn this task.

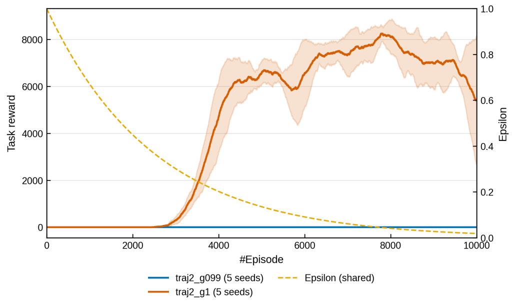

Figure 4.11. 2-phase task. Training task reward of the biased learner with $\gamma_s = 0.99$ and $\gamma_s = 1$. The dashed curve shows ϵ.

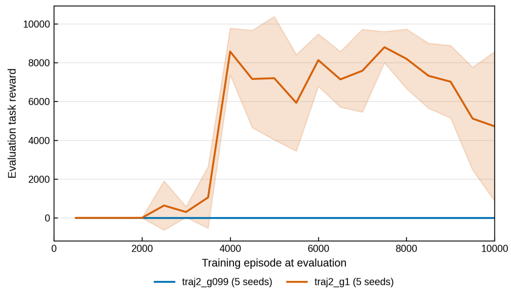

Figure 4.12. 2-phase task. Periodic greedy evaluation of the biased policy with $\gamma_s = 0.99$ and $\gamma_s = 1$.

γ_s	**Best success rate (%)**	**Same-state fraction (%)**	**Same-state reward per transition**	**Same-state reward per episode**
0.99	0.00 ± 0.00	82.39 ± 0.96	-89.51 ± 0.05	-5000.60 ± 53.21
1.00	96.94 ± 1.91	91.48 ± 0.22	0.00 ± 0.00	0.00 ± 0.00

Table 4.5. Comparison of shaping discounts on the 2-phase task: best-policy success rate and same-state transition statistics.

The results show a clear difference between the two shaping configurations. With $\gamma_s = 0.99$, none of the five runs is able to learn a successful policy, whereas setting $\gamma_s = 1$ leads to a best evaluation success rate of $96.94\% \pm 1.91\%$. Furthermore, in both configurations, we notice that most of the time (expressed in transitions) the agent performs actions remaining in states mapped to the same abstract one. This fact is strictly related to the ground environment dynamics and the abstraction adopted over it. A grid abstraction does not capture any detail about the physics of the lander, intentionally. Hence, after each transition, with high probability the agent remains in the same abstract cell.

Let's consider the situation in which the abstract cell does not change after a transition. Formally, the state s and s' involved in the shaping formula are the same. Then:

$$F(s, a, s') = (\gamma_s - 1)V(s)$$

Therefore, every transition receives a negative shaping signal whenever $V(s) > 0$ and $\gamma_s < 1$. This generates a high cumulative penalty over a single episode when $\gamma_s = 0.99$. Instead, with $\gamma_s = 1$, we neutralize this effect on the reward signal.

Overall, this behavior can be interpreted as a temporal misalignment between the ground MDP and the abstract model.

Ground-level transitions occur at a much finer temporal resolution than transitions that effectively change the abstract state. Consequently, several consecutive ground interactions may correspond to the same abstract cell and therefore to the same potential value. This effect leads to a non-informative shaping guidance for the agent.

4.3.3 Discussion

The experiments presented in this section highlight two practical aspect of the framework related to its application on a complex domain.

First, the coupled architecture biased-unbiased learner does not reliably transfer the benefits obtained by the biased policy to the unbiased one. The results suggest that this behavior is strictly related to the data distribution induced by the biased learner. In fact, on a very simple task like the *1-phase trajectory*, the unbiased standalone learner was able to learn a successful policy. Hence, for this task, we can exclude that the problem was the sparse-reward setting. However, for more complex task, the shaping guidance is still necessary.

The second issue is about the temporal misalignment between the ground domain and the abstract one. More generally, this issue can happen between two consecutive layers in the hierarchy. Intentionally, in this thesis, we have adopted a very trivial abstraction over a complex environment for the following reasons:

- at this moment, we do not have any automatic mechanism to design the abstraction levels;
- in this setting, it was not clear how to integrate the complex physics of *Lunar Lander* into the abstraction levels.

In particular, the temporally misalignment emerges when the changing state frequency between two consecutive levels is very different. As happened in the current setting, this could lead to the generation of artificial penalties received by the agent, also due to the mathematical structure of the shaping formula.

From these considerations, we continue the experimental evaluation process adopting the following choices:

- we will evaluate the performance over the various tasks considering the biased learner at the ground MDP.
  In [6], this choice was already adopted when the authors studied the framework on a robotic domain. In particular, they says: "*In practice, in the considered robot control problems (modeled as Goal MDPs), the main challenge is to compute a policy achieving the goal with high sample efficiency, rather than finding an optimal one.*"[1];
- we will set $\gamma_s = 1$ to neutralize the temporal misalignment consequences.

[1]Extracted from [6], Section 7, page 7.

Theoretical consequences of the assumptions

First of all, by considering the biased learner as the final ground-level policy, we accept that the learned policy may differ from the optimal one induced by the original reward function. We can accept the risk because we have guarantee that the abstraction provided are synchronized and coherent with the task specification on the ground MDP.

Furthermore, adopting $\gamma_s = 1$, we lose the PBRS policy invariance property. This guarantee was already relaxed in [6]. However, this choice is asymmetrical, in the sense that the discount factor γ is different from γ_s.
Hence, it is no longer true that

$$G = \sum_{t=0}^{T-1} \gamma^t \left(\gamma V(s_{t+1}) - V(s_t)\right) = -V(s_0) + \gamma^T V(s_T)$$

because the new formulation is

$$G = \sum_{t=0}^{T-1} \gamma^t \left(\gamma_s V(s_{t+1}) - V(s_t)\right) = \sum_{t=0}^{T-1} \gamma^t \left(V(s_{t+1}) - V(s_t)\right)$$

and the intermediate potential terms no longer cancel in the sum.
Therefore, the cumulative shaping return depends also on the intermediate potential values visited along the trajectory.

4.4 RQ2: Performance under Increasing Temporal Complexity

The experiments shown in this section investigates how the framework behaves when the temporal complexity of the task increases.

The *2-phase trajectory* introduced in the previous section represents the starting point of this analysis. From this task specification, we increase the task complexity by requiring the satisfaction of longer and more structured trajectories to the agent. More precisely, the task specifications considered in this section are:

- *2-phase trajectory*;
- *3-phase trajectory* in three different versions: simple, hard and extreme;
- *4-phase trajectory*;
- *5-phase trajectory*.

Each task will be tested using a single abstraction level over the ground MDP, solved with Value Iteration. In particular, the grids used in these experiments are:

$$6 \times 6 \quad 12 \times 12 \quad 24 \times 24 \quad 48 \times 48 \quad 180 \times 180 \quad 360 \times 360$$

These settings allow us to investigate if there exist a sort of correlation between the grid precision and the task complexity. In other words, we are investigating if more precise grids provide a better guidance useful to fulfill more difficult task.

4.4.1 2-Phase Trajectory

The first task considered is the *2-phase trajectory*, already presented in Section 4.3.1.

Hyperparameters

Here, are reported the common hyperparameters adopted for these experiments.

Parameter	**Value**
Episodes	8000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	10
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes during learning	1000
Eval. episodes after learning	1000

Table 4.6. Hyperparameters adopted in the experiments.

Results and Interpretations

We evaluate these experiments considering the learning curves and the evaluation performances. In particular, we will consider both the evaluation performances during the training phase and after it, with different evaluation seeds.

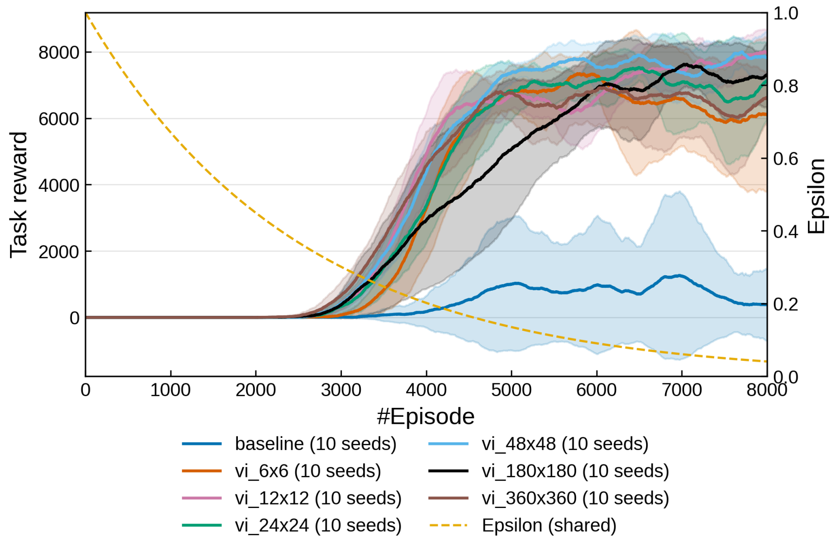

Figure 4.13. 2-phase trajectory. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

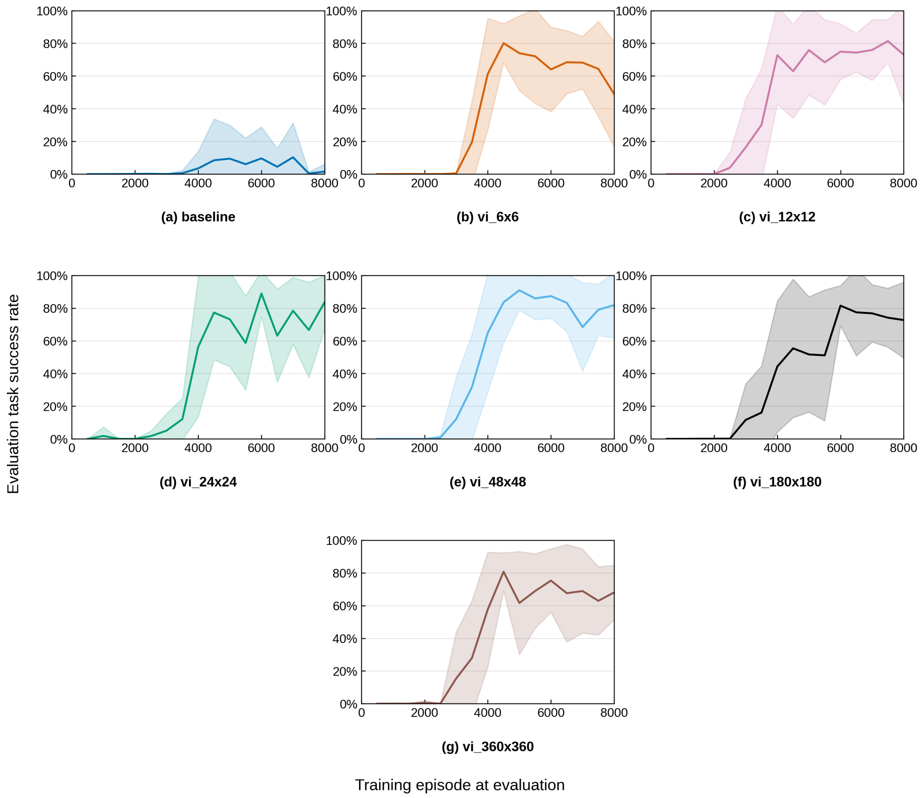

Figure 4.14. 2-phase trajectory. Periodic evaluation success rate during training for the six abstraction grid resolutions.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$16.10 \pm 30.52\,\%$	$1.99 \pm 5.97\,\%$
6×6	$91.89 \pm 10.15\,\%$	$49.20 \pm 31.81\,\%$
12×12	$97.42 \pm 1.83\,\%$	$73.53 \pm 29.99\,\%$
24×24	$96.81 \pm 3.71\,\%$	$84.33 \pm 15.18\,\%$
48×48	$98.26 \pm 0.99\,\%$	$82.55 \pm 19.05\,\%$
180×180	$95.08 \pm 7.10\,\%$	$73.07 \pm 23.50\,\%$
360×360	$97.01 \pm 2.25\,\%$	$69.38 \pm 17.28\,\%$

Table 4.7. Post-learning evaluation for Trajectory 2. Success rates are reported as mean $\pm$ standard deviation across 10 seeds, with each policy evaluated over 1000 episodes.

The learning curves highlight the importance of the abstraction-based guidance. The baseline configuration, without any shaping signal, is not able to reliably learn a successful policy that solves the task. On the other hand, all others configurations were capable to find the policy.

Furthermore, Figure 4.13 exhibits a largely comparable learning dynamics across different grid resolutions. At this moment, no clear relationship emerges between grid precision and quality of the obtained policy. This fact is also confirmed by the post-learning evaluation, in which all abstraction-based best policies reach a success rate greater than 90%.

Analyzing the variance of the periodically evaluations performed during the training phase and noticing the differences between the performances of best and last policies, we have an empirical support that the best policy selection mechanism is crucial. In each setting, the last policy performs worst if compared to the correspondent best policy.

4.4.2 3-Phase Trajectory

This task corresponds to a trajectory composed of three phases: reaching the first waypoint, then the second waypoint, and finally the target region. The LTL$_f$ formula is:

$$\varphi = \Diamond\left(\mathrm{wp}_1 \wedge \circ\left(\Diamond\left(\mathrm{wp}_2 \wedge \circ\left(\Diamond\left(\mathrm{wp}_3 \wedge \circ\left(\Diamond g_1\right)\right)\right)\right)\right)\right)$$

where wp_1 and wp_2 are the waypoint predicates, and g_1 is the final target predicate.

The corresponding automaton is the one depicted in the following figure.

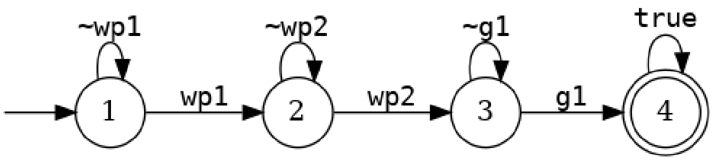

Figure 4.15. DFA for the 3-phase trajectory.

Task Variants

The same temporal specification is tested with three different spatial configurations. In particular, the differences between these version corresponds to the second waypoint's spatial position, as depicted in the following figures.

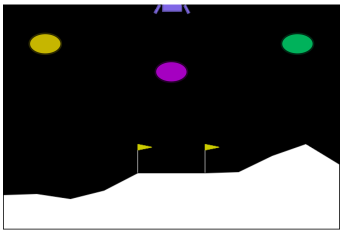

Figure 4.16. 3-phase trajectory: Simple.

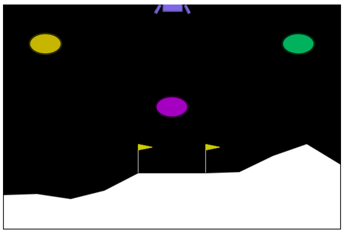

Figure 4.17. 3-phase trajectory: Hard.

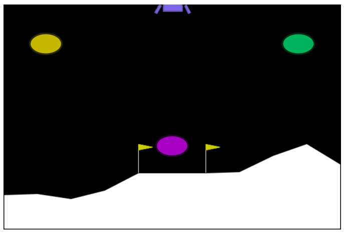

Figure 4.18. 3-phase trajectory: Extreme.

As we can see, the idea is to increase the difficulty associated to the required trajectory in order to stress the framework evaluating the resulting policies.

Hyperparameters

The hyperparameters adopted for these experiments are the same as those reported in Section 4.4.1. They are shared for all three variants of the task.

Training Results – Simple

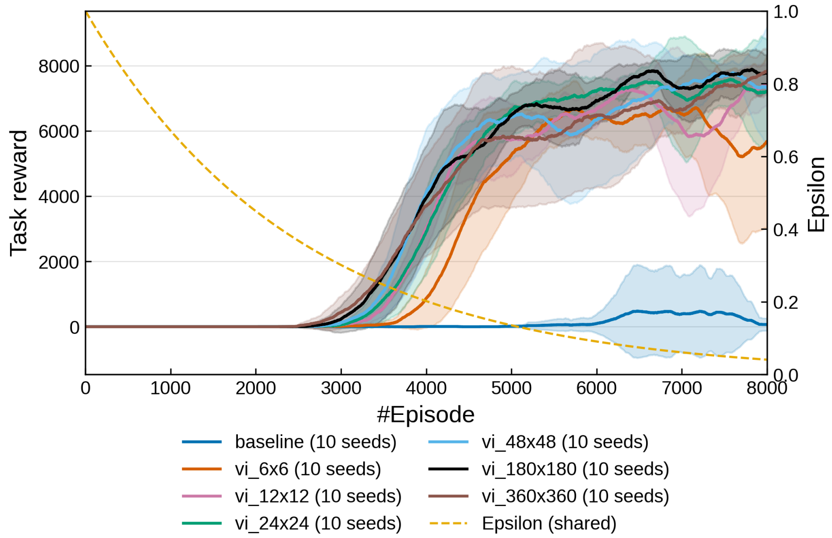

Figure 4.19. 3-phase trajectory – Simple. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

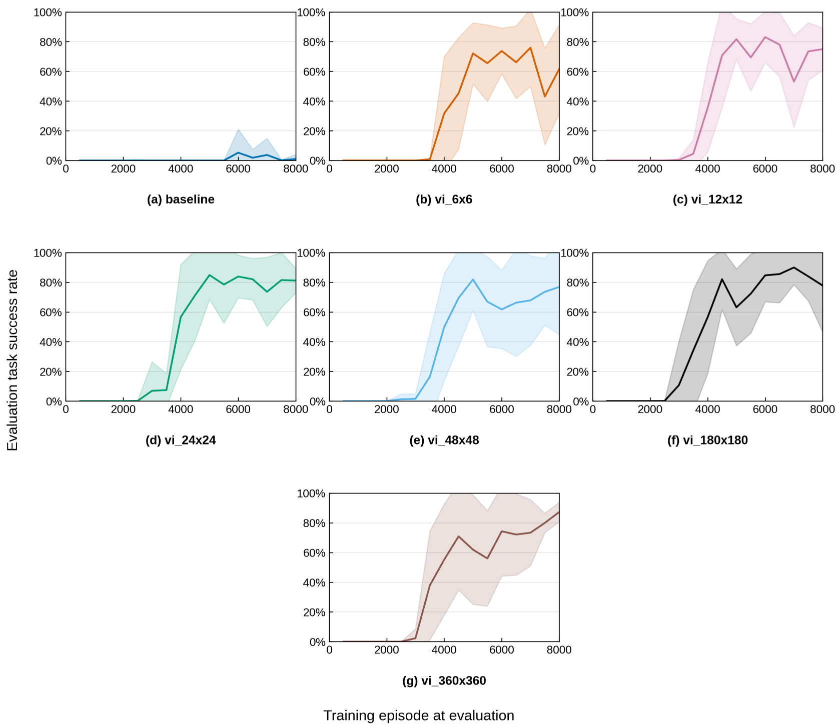

Figure 4.20. 3-phase trajectory – Simple. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Training Results – Hard

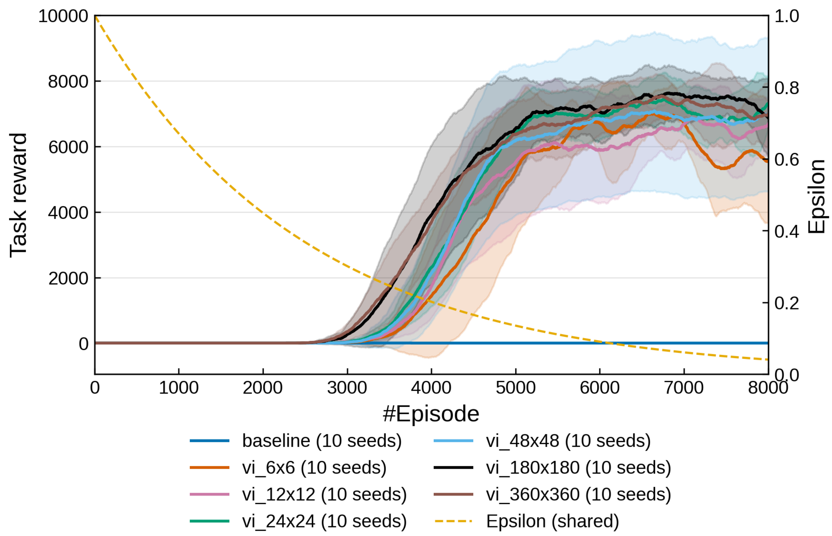

Figure 4.21. 3-phase trajectory – Hard. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

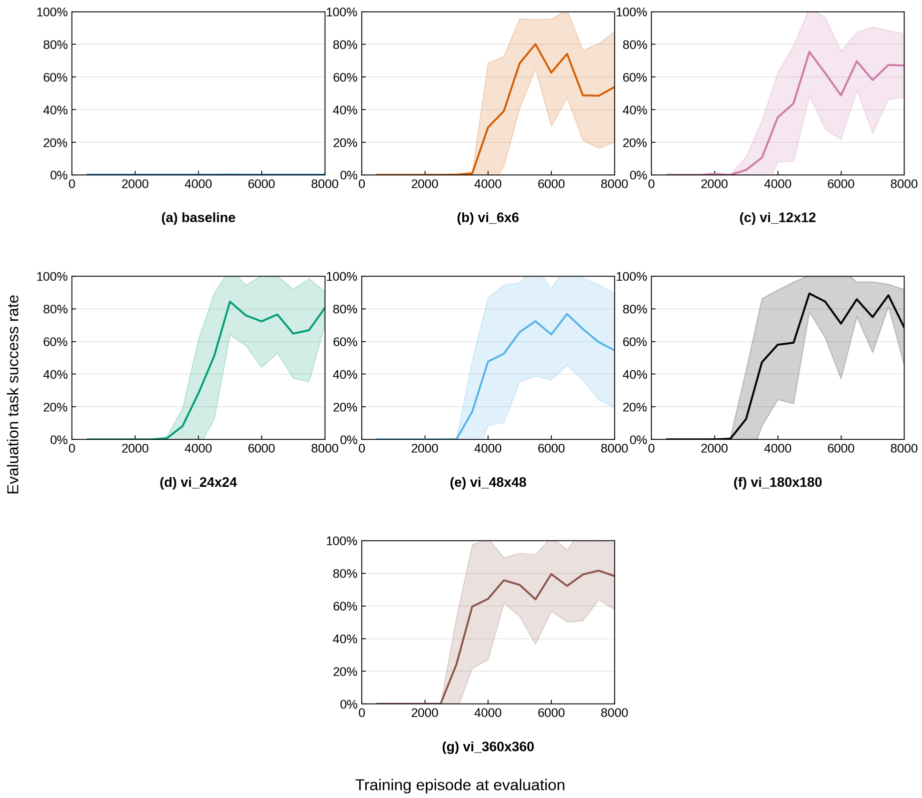

Figure 4.22. 3-phase trajectory – Hard. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Training Results – Extreme

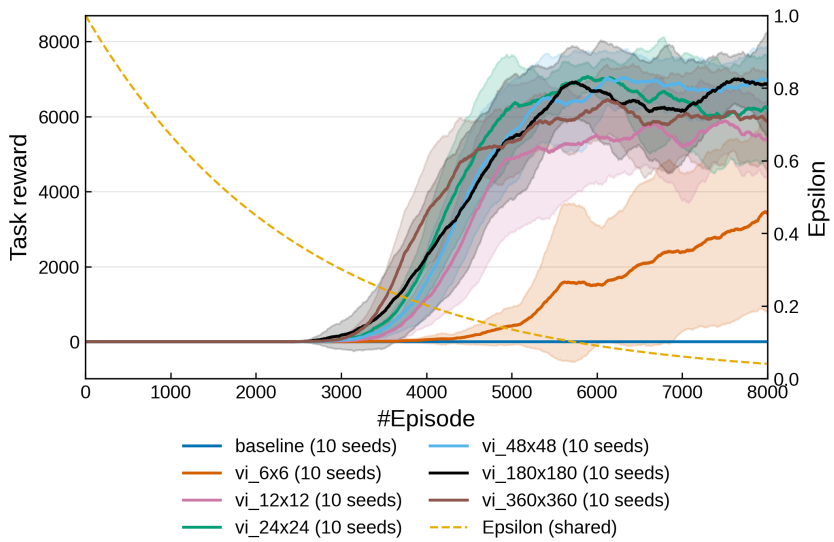

Figure 4.23. 3-phase trajectory – Extreme. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

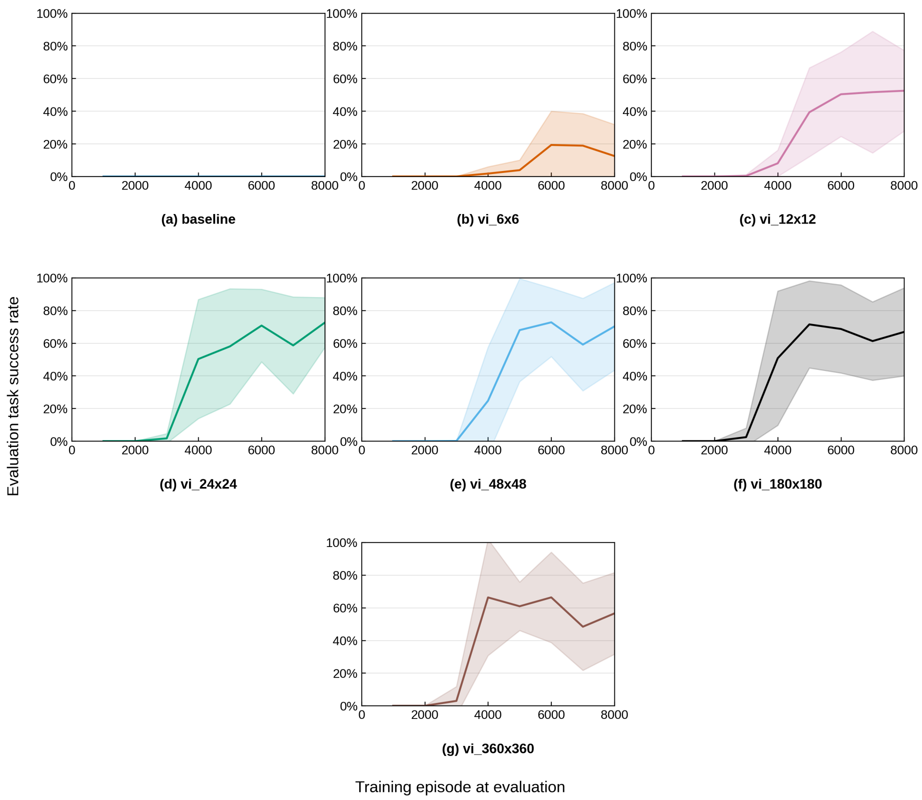

Figure 4.24. 3-phase trajectory – Extreme. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Post-Training Evaluation

Abstraction	Best policy success rate	Last policy success rate
Baseline	$5.3 \pm 15.9\,\%$	$0.9 \pm 2.7\,\%$
6×6	$95.0 \pm 2.3\,\%$	$62.7 \pm 30.5\,\%$
12×12	$94.1 \pm 5.6\,\%$	$75.3 \pm 14.6\,\%$
24×24	$98.2 \pm 1.3\,\%$	$82.4 \pm 8.0\,\%$
48×48	$98.3 \pm 1.6\,\%$	$76.9 \pm 32.5\,\%$
180×180	$98.8 \pm 1.8\,\%$	$78.2 \pm 31.4\,\%$
360×360	$96.5 \pm 2.1\,\%$	$87.5 \pm 5.8\,\%$

Table 4.8. Post-training evaluation results on the Simple variant. Values report the mean success rate and standard deviation across 10 training seeds, using 1000 evaluation episodes per policy.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$0.1 \pm 0.4\,\%$	$0.0 \pm 0.0\,\%$
6×6	$95.0 \pm 6.1\,\%$	$54.6 \pm 34.0\,\%$
12×12	$93.1 \pm 4.3\,\%$	$67.7 \pm 19.4\,\%$
24×24	$97.0 \pm 2.5\,\%$	$81.3 \pm 9.1\,\%$
48×48	$88.0 \pm 28.6\,\%$	$55.0 \pm 35.1\,\%$
180×180	$98.6 \pm 2.0\,\%$	$68.9 \pm 22.9\,\%$
360×360	$97.1 \pm 2.9\,\%$	$78.2 \pm 20.7\,\%$

Table 4.9. Post-training evaluation results on the Hard variant. Values report the mean success rate and standard deviation across 10 training seeds, using 1000 evaluation episodes per policy.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$0.0 \pm 0.0\,\%$	$0.0 \pm 0.0\,\%$
6×6	$28.3 \pm 23.9\,\%$	$12.4 \pm 19.7\,\%$
12×12	$77.8 \pm 11.3\,\%$	$53.1 \pm 24.9\,\%$
24×24	$93.2 \pm 5.3\,\%$	$73.7 \pm 15.1\,\%$
48×48	$87.6 \pm 13.0\,\%$	$70.1 \pm 26.9\,\%$
180×180	$90.1 \pm 11.2\,\%$	$67.3 \pm 26.9\,\%$
360×360	$91.6 \pm 5.3\,\%$	$58.2 \pm 25.1\,\%$

Table 4.10. Post-training evaluation results on the Extreme variant. Values report the mean success rate and standard deviation across 10 training seeds, using 1000 evaluation episodes per policy.

Results Interpretation

The results show that the abstraction-based guidance is still effective for all the spatial configuration tested, consistently outperforming the baseline.

For the *Simple* and *Hard* variants, most grid resolutions lead to similar final performance. Nevertheless, from the learning curves, we notice that the ones associated with a finer abstraction tend to reach the plateau earlier during the training phase. The trend becomes more evident in the *Extreme* variant, where the coarsest resolutions like the 6×6 and 12×12 learn slowly and achieve lower performance. This is also confirmed from the periodically greedy evaluations performed during training.

Analyzing more in depth the post-training evaluation results about the *Hard* variant, it emerges that the worst configuration (apart from the baseline) involves the 48×48 abstraction grid. In reality, the results shown for this configuration are strongly influenced by a single seed in which the learner completely failed to discover a successful policy. The other runs achieves performances comparable to the ones obtained with other abstraction grids. The presence of this outlier raises an alarm about the guarantee to find an useful policy. The abstraction built over the ground MDP cannot provide to us the full certainty that the agent will discover the required trajectory.

Since the three variants share the same temporal specification, the observed differences cannot be linked to the temporal complexity itself, but rather to the increasing spatial difficulty of the required trajectory. Overall, the results suggest that there is a benefit of using a more precise abstraction and this becomes more relevant when finer spatial information is required to complete the task. Nevertheless, the relation between grid precision and policy's quality do not appears monotonic. Rather, the results suggest the existence of a task-dependent minimum resolution. Below this threshold, the abstraction may be too coarse to provide a sufficiently informative guidance.

4.4.3 4-Phase Trajectory

This task corresponds to a trajectory composed of four phases: reaching the first waypoint, then the second waypoint, then the third waypoint and finally the target region. The LTL_f formula is:

$$\varphi = \Diamond\,(\mathrm{wp}_1 \wedge \circ\,(\Diamond\,(\mathrm{wp}_2 \wedge \circ\,(\Diamond\,(\mathrm{wp}_3 \wedge \circ\,(\Diamond g_1))))))$$

where wp_1, wp_2 and wp_3 are the waypoint predicates, and g_1 is the final target predicate.

The corresponding automaton is the one depicted in the following figure

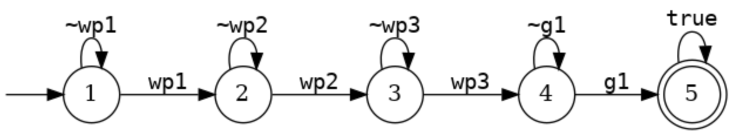

Figure 4.25. DFA for the 4-phase task.

and the required trajectory is:

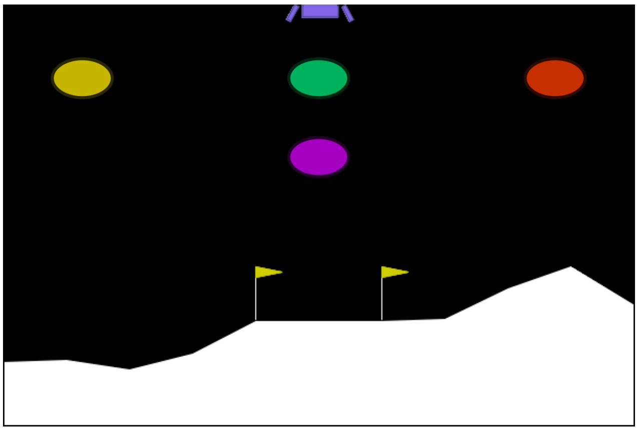

Figure 4.26. 4-phase task: yellow → purple → green → red.

Hyperparameters

Here, are reported the common hyperparameters adopted for these experiments.

Parameter	Value
Episodes	12000
ϵ decay	0.9998
ϵ minimum	0.01
Seeds	10
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes during learning	1000
Eval. episodes after learning	1000

Table 4.11. Hyperparameters adopted in the experiments.

Results and Interpretations

As a first stage, we evaluate these experiments considering the learning curves and the evaluation performances. In particular, we will consider both the evaluation performances during the training phase and after it, with different evaluation seeds. Then, we move on analyzing more in depth the results obtained for each abstraction grid, due to the presence of some outlier seeds.

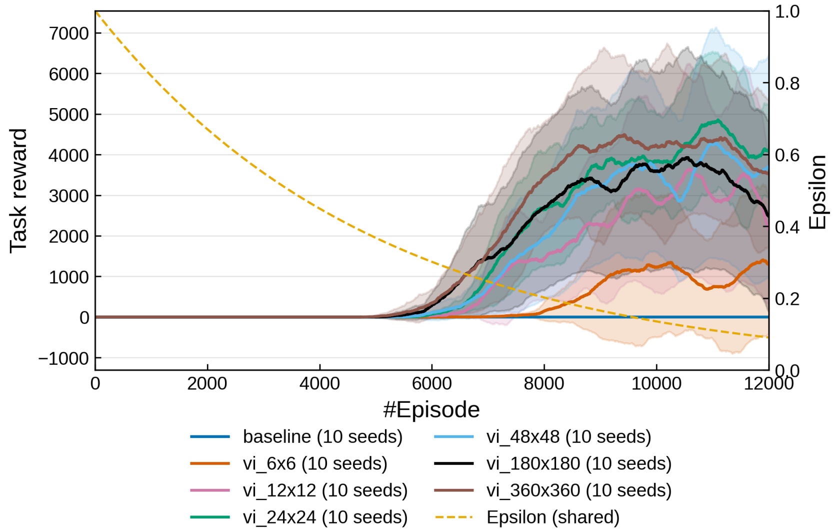

Figure 4.27. 4-phase trajectory. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

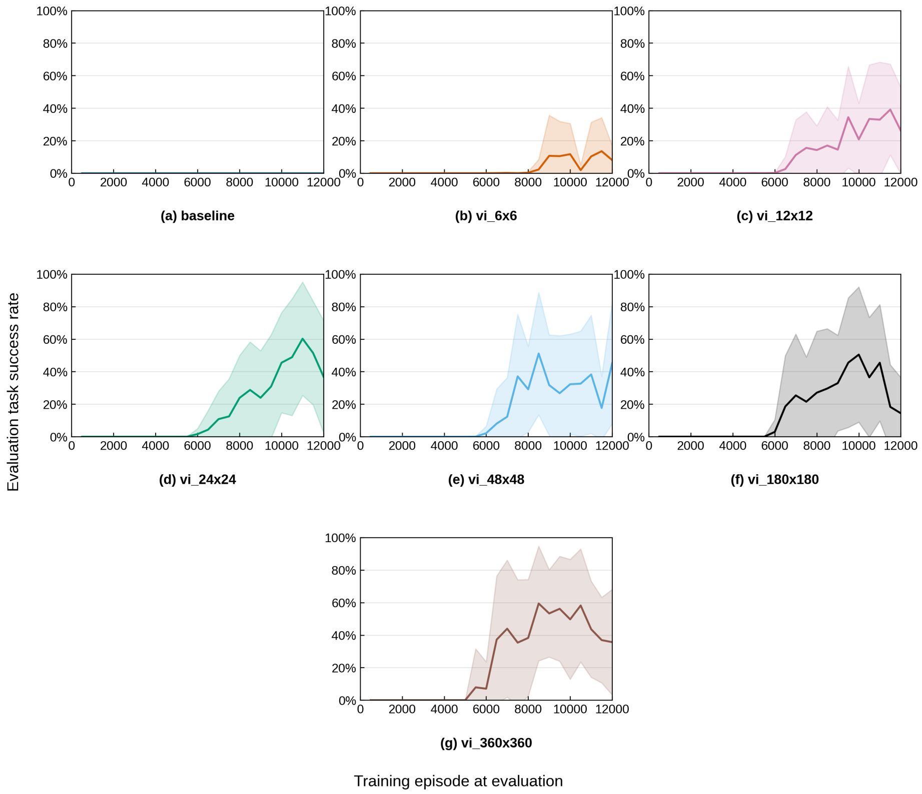

Figure 4.28. 4-phase trajectory. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$0.0 \pm 0.0\,\%$	$0.0 \pm 0.0\,\%$
6×6	$24.8 \pm 31.7\,\%$	$8.1 \pm 9.2\,\%$
12×12	$53.5 \pm 37.3\,\%$	$26.1 \pm 26.1\,\%$
24×24	$83.6 \pm 15.0\,\%$	$36.8 \pm 34.5\,\%$
48×48	$74.9 \pm 37.6\,\%$	$45.9 \pm 37.6\,\%$
180×180	$64.2 \pm 42.3\,\%$	$14.7 \pm 21.6\,\%$
360×360	$85.1 \pm 28.6\,\%$	$35.7 \pm 32.8\,\%$

Table 4.12. Four-phase trajectory. Post-training evaluation results for the best and last policies over ten training seeds and 1000 evaluation episodes per policy.

The results presented so far reveal a significant increase in the variance characterizing the performance of the best policies, as shown in Table 4.12. A deep analysis of the individual runs shows that the variance obtained is strongly influenced by the presence of outlier seeds. The presence of such cases is not completely unexpected, considering that their existence has already been discovered in the *3-phase* experiments and that the new task is more difficult if compared to the previous one. For this reason, the mean success rate alone is no longer enough to correctly characterize the performances of the policies obtained. Hence, we move on in the analysis studying the performance obtained after excluding the identified outliers and investigating the differences between successful and failed training runs.

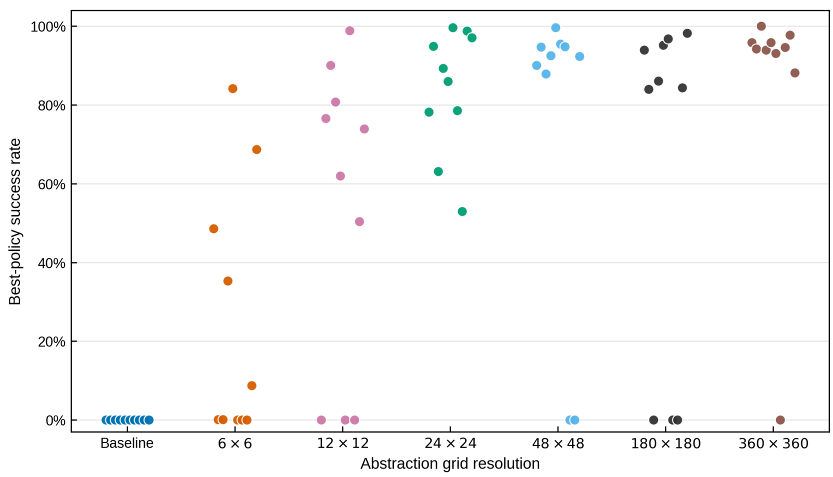

Figure 4.29. 4-phase trajectory. Distribution of post-training best-policy success rates for the baseline and the six abstraction grid resolutions. Each point represents one training seed, with each policy evaluated over 1000 episodes.

Figure 4.29 confirms that the high-variance observed is strongly influenced by the presence of unsuccessful runs. It is also evident that when we augment the grid precision there is a general improvement of the quality. In fact, for finer abstractions, successful policies are concentrated close to the upper part of the success rate range (excluding the failed runs). But, the lower performance obtained with a coarser abstraction grid cannot be explained only by the existence of failed runs. From the plot it emerges that the policy associated to these grids are spread across a

wide range of success rates, suggesting that their guidance is less informative and consistent to fulfill the more demanding task.
At this point, it is interesting to study the differences between successful and failed runs.

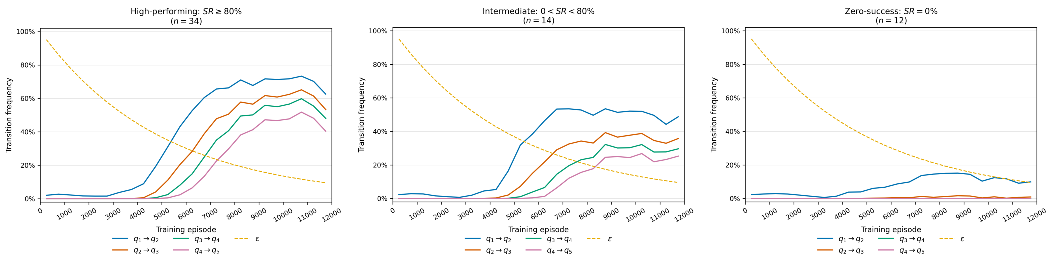

Figure 4.30. 4-phase trajectory. DFA-transition frequencies during training, grouped by the post-training success rate of the best policy: high-performing ($SR \geq 80\%$), intermediate ($0 < SR < 80\%$), and zero-success ($SR = 0\%$) runs. Each curve reports the fraction of episodes containing the corresponding transition, computed over non-overlapping windows of 500 episodes and averaged across the runs in each group.

From the plots of Figure 4.30 we notice a clear distinction among the three groups. The successful policies are characterized by a progressive increase in all the relevant DFA transitions. This indicates that successful policies gradually learn to complete increasingly longer prefixes of the required trajectory until the whole temporal task is reliably satisfied. This behavior is not unexpected because the task is sequential and each episode start from the initial DFA state.
Intermediate policies show a similar but a weaker pattern.
On the other hand, the failed runs exhibit a completely different behavior. In this case, only the first transition appears with a limited frequency, whereas the subsequent ones remain almost absent throughout training.

The abstraction itself does not appear the main explanation in failed runs. In fact, we have obtained counterexamples in which the same abstraction configuration is able to produce successful policies (we recall that without abstraction this task appears too hard to be solved).
A possible explanation is strictly related to exploration during the learning phase. In a purely sequential task, the agent always starts from the initial phase and can discover a subsequent DFA transition only after successfully completing all the preceding ones. Therefore, if a given phase of the required trajectory is not discovered sufficiently often during training, all the subsequent phases become unreachable.
This effect is particularly important when exploration decreases over time. It can be noticed also from the first plot of Figure 4.30. By the time the agent has consolidated the first phase of the task (between 6000 and 7000 episodes), the exploration rate has already decreased to approximately 30%. Therefore, only a limited amount of exploratory behavior is still available to discover and reinforce all the subsequent phases of the required trajectory.

Therefore, these considerations suggest that the interaction between sequential task and exploration dynamics is a possible explanation for the unsuccessful runs.

However, this interpretation is still an hypothesis and requires further investigation.

4.4.4 5-Phase Trajectory

This task corresponds to a trajectory composed of five phases: reaching the first waypoint, then the second, third and fourth waypoints in sequence, and finally the target region. The LTL$_f$ formula is:

$$\varphi = \Diamond\left(\mathrm{wp}_1 \wedge \circ\left(\Diamond\left(\mathrm{wp}_2 \wedge \circ\left(\Diamond\left(\mathrm{wp}_3 \wedge \circ\left(\Diamond\left(\mathrm{wp}_4 \wedge \circ\left(\Diamond g_1\right)\right)\right)\right)\right)\right)\right)\right)$$

where wp_1, wp_2, wp_3 and wp_4 are the waypoint predicates, and g_1 is the final target predicate.

The corresponding automaton is depicted in the following figure.

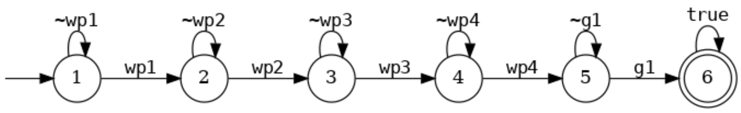

Figure 4.31. DFA for the 5-phase task.

The required trajectory is shown below.

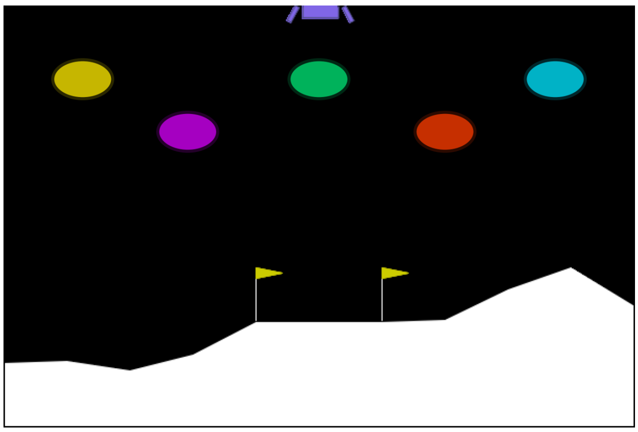

Figure 4.32. 5-phase task: yellow → purple → green → red → blue.

Hyperparameters

The hyperparameters adopted for these experiments are the same as those reported in Section 4.4.3.

Results and Interpretations

As in the previous experiment, we start analyzing the learning curves and the evaluation performances. Then, we move on investingating about the individual runs in order to better interpret the effect of high variance and unsuccessful seeds.

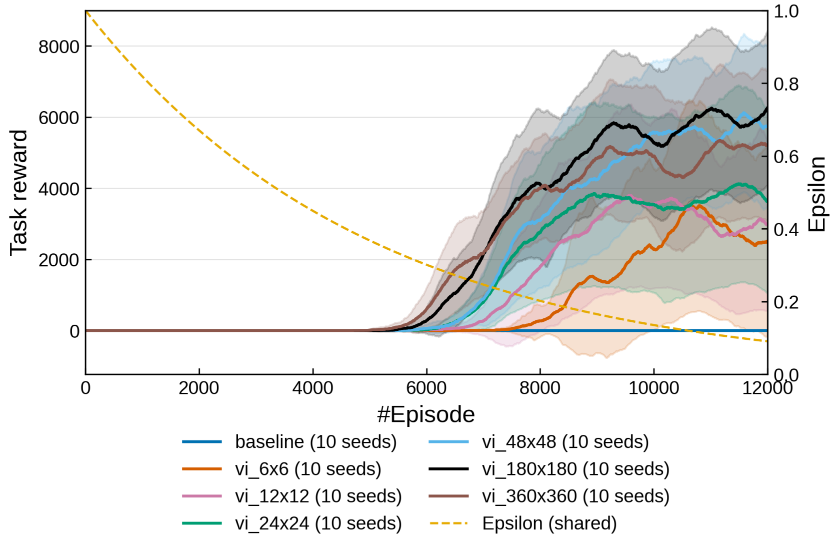

Figure 4.33. 5-phase trajectory. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

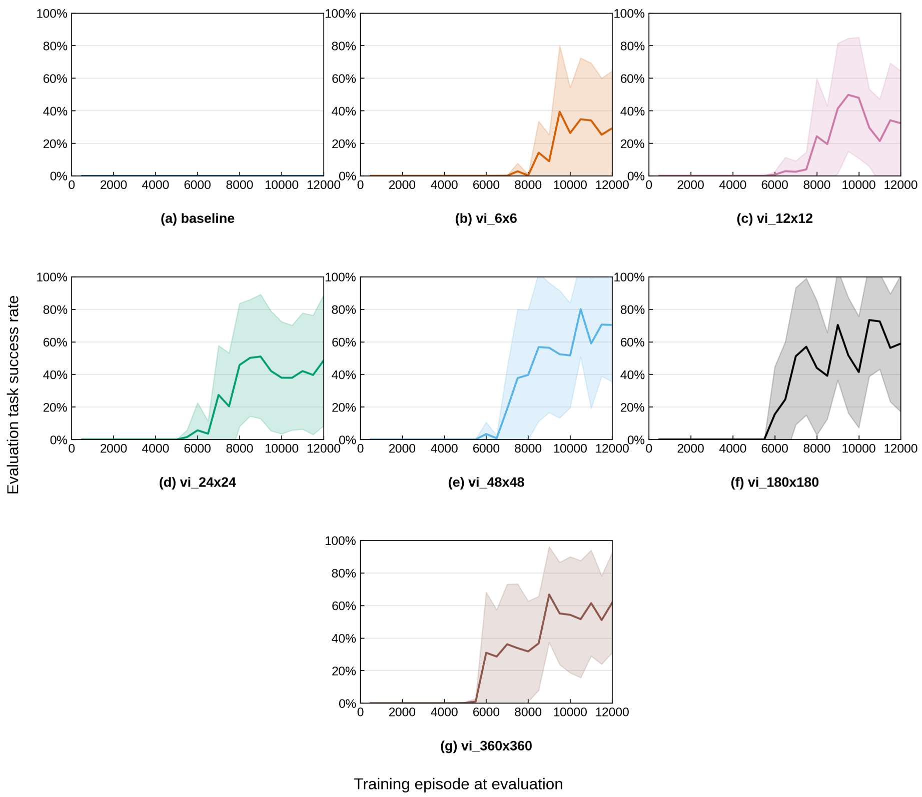

Figure 4.34. 5-phase trajectory. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$0.0 \pm 0.0\,\%$	$0.0 \pm 0.0\,\%$
6×6	$52.3 \pm 43.6\,\%$	$29.2 \pm 35.1\,\%$
12×12	$62.3 \pm 42.3\,\%$	$32.3 \pm 32.2\,\%$
24×24	$65.2 \pm 43.2\,\%$	$48.5 \pm 40.2\,\%$
48×48	$87.3 \pm 29.4\,\%$	$70.4 \pm 34.9\,\%$
180×180	$88.5 \pm 29.5\,\%$	$58.9 \pm 41.9\,\%$
360×360	$85.6 \pm 28.7\,\%$	$61.8 \pm 30.7\,\%$

Table 4.13. Five-phase trajectory. Post-training evaluation results for the best and last policies over ten training seeds and 1000 evaluation episodes per policy.

From these results, the same initial considerations done for the previous task are still valid. However, the distinction between the policies obtained with a coarse or a fine abstraction appears to be more evident. In particular, the 48×48, 180×180, and 360×360 configurations achieve higher best-policy success rates, while the coarser grids remain characterized by lower performance and large variability. Moreover, unexpectedly, the additional temporal complexity for this task does not result in a degradation if we compare the performance obtained with the previous one.

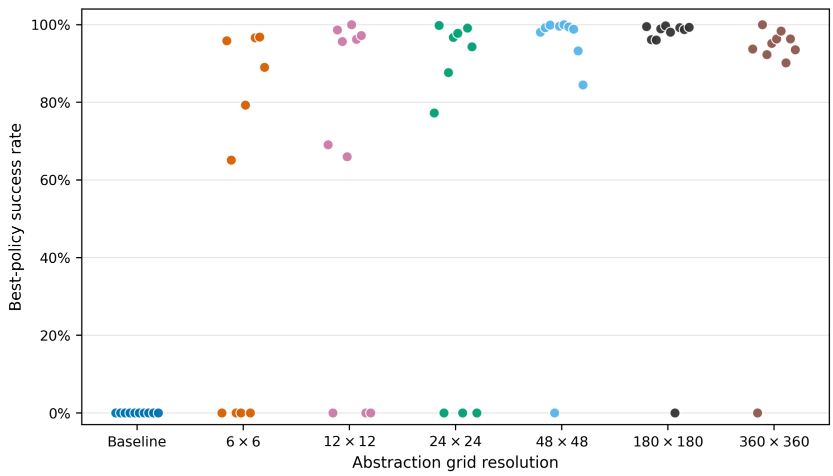

Figure 4.35. 5-phase trajectory. Distribution of post-training best-policy success rates for the baseline and the six abstraction grid resolutions. Each point represents one training seed, with each policy evaluated over 1000 episodes.

Another time, the analysis of single runs provide a confirm that the high variance is influenced by failed runs. We notice also the same trend described for the previous task, in which finer abstraction grid seems to be more reliable rather than the coarse ones.

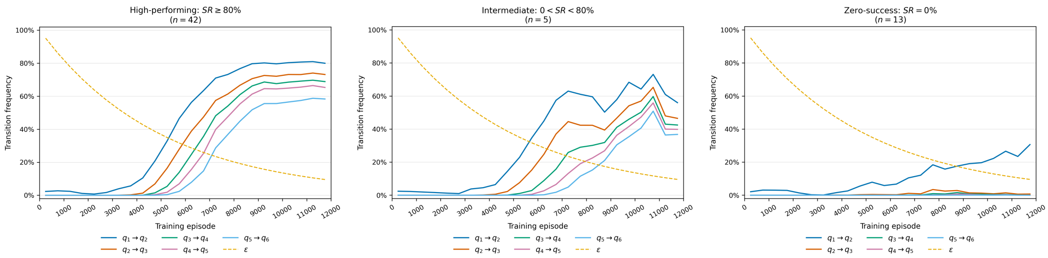

Figure 4.36. 5-phase trajectory. DFA-transition frequencies during training, grouped by the post-training success rate of the best policy: high-performing ($SR \geq 80\%$), intermediate ($0 < SR < 80\%$), and zero-success ($SR = 0\%$) runs. Each curve reports the fraction of episodes containing the corresponding transition, computed over non-overlapping windows of 500 episodes and averaged across the runs in each group.

Also the DFA transition analysis exhibits the same pattern: for successful runs the policies are built discovering sequentially the several phases of the required task. Moreover, we notice some delay in the consolidation of the first phase that now starts between 8000 and 9000 episodes. Despite the fact that the first phase required by this task is the same of the previous ones, this delay suggests that the consolidation of the first phases may not be completely independent of the subsequent task structure.

4.4.5 Discussion

Overall, the experiments shown in this section exhibit that the framework is able to support the learning of increasingly complex episodic non-Markovian tasks in the *Lunar Lander* domain. The abstraction-based shaping signal provides a substantial advantage over the baseline without guidance, which is not able to solve the required trajectories.

At the same time, the results highlight that increasing temporal complexity mainly affects the reliability of the learning process rather than imposing a limitation on the quality of the policies that can be obtained. In particular, for more demanding tasks, different training seeds may lead to very different results, ranging from highly successful policies to complete failures. The analysis of single runs and DFA progressions suggests that this behavior could be related to the sequential nature of the task considered, but also to the exploration strategy adopted during the learning phase. In the more complex settings, the full trajectory is not always discovered during exploration, since reaching a later phase requires the agent to have already discovered and sufficiently consolidated all the preceding ones.

Nevertheless, the results show that the abstraction built over the ground MDP was crucial to solve the tasks proposed. Starting from the simpler one, we notice that even coarse abstraction grid were sufficient to obtain an useful policy that fulfills the objective required. The relationship between grid precision and task feasibility is not monotonic. However, as the task becomes more demanding, finer abstractions lead to more reliable results. Increasing the precision of the abstraction also increases the complexity of the abstract decision problem. In this context, this issue was partially hidden by the fact that the abstract grids are solved through

Value Iteration, exploiting a full-known MDP. This assumption is not general and could not be feasible when richer or higher-dimensional abstractions are considered, especially if we are trying to adopt this framework in a complex and real scenario.

4.5 RQ3: Effect of Multi-Level Abstraction

4.5.1 Experimental Design

4.5.2 Learning Performance

4.5.3 Analysis of the Guidance Signal

4.5.4 Effect of Task Complexity on Multi-Level Guidance

4.5.5 Discussion

4.6 RQ4: Effectiveness on Cyclic Temporal Tasks

4.6.1 Task Definition and Cyclic Automaton

4.6.2 Experimental Design

4.6.3 Learning Behaviour and Results

4.6.4 Discussion

4.7 Additional Case Studies

4.8 Overall Discussion

Chapter 5

Conclusions and Future Work

5.1 Summary of the Approach

5.2 Main Findings

5.3 Contributions

5.4 Limitations

5.5 Future Research Directions

5.6 Concluding Remarks

Bibliography

[1] Abel, D., Umbanhowar, N., Khetarpal, K., Arumugam, D., Precup, D., and Littman, M. Value preserving state-action abstractions. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics* (edited by S. Chiappa and R. Calandra), vol. 108 of *Proceedings of Machine Learning Research*, pp. 1639–1650. PMLR (2020). Available from: `https://proceedings.mlr.press/v108/abel20a.html`.

[2] Ausiello, G., D'Amore, F., and Gambosi, G. *Linguaggi, modelli, complessità*. Franco Angeli (2003). ISBN 9788846444707.

[3] Bacchus, F., Boutilier, C., and Grove, A. Rewarding behaviors. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence*, AAAI'96, p. 1160–1167. AAAI Press (1996).

[4] Brafman, R. I., Giacomo, G. D., and Patrizi, F. Ltlf/ldlf non-markovian rewards. In *AAAI Conference on Artificial Intelligence* (2018).

[5] Canonaco, G., Ardon, L., Pozanco, A., and Borrajo, D. On the sample efficiency of abstractions and potential-based reward shaping in reinforcement learning (2025). Available from: `https://arxiv.org/abs/2404.07826`, `arXiv:2404.07826`.

[6] Cipollone, R., Favorito, M., Maiorana, F., De Giacomo, G., Iocchi, L., and Patrizi, F. Exploiting robot abstractions in episodic rl via reward shaping and heuristics. *Robotics and Autonomous Systems*, **193** (2025), 105116. `doi:10.1016/j.robot.2025.105116`.

[7] De Giacomo, G. and Vardi, M. Linear temporal logic and linear dynamic logic on finite traces. *IJCAI International Joint Conference on Artificial Intelligence*, (2013), 854.

[8] Dietterich, T. G. Hierarchical reinforcement learning with the maxq value function decomposition (1999). Available from: `https://arxiv.org/abs/cs/9905014`, `arXiv:cs/9905014`.

[9] Mnih, V., et al. Human-level control through deep reinforcement learning. *Nature*, **518** (2015), 529. `doi:10.1038/nature14236`.

[10] Ng, A. Y., Harada, D., and Russell, S. J. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings*

of the Sixteenth International Conference on Machine Learning, ICML '99, p. 278–287. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1999). ISBN 1558606122.

[11] Parr, R. and Russell, S. Reinforcement learning with hierarchies of machines. In *Advances in Neural Information Processing Systems* (edited by M. Jordan, M. Kearns, and S. Solla), vol. 10. MIT Press (1997). Available from: `https://proceedings.neurips.cc/paper_files/paper/1997/file/5ca3e9b122f61f8f06494c97b1afccf3-Paper.pdf`.

[12] Pnueli, A. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*, pp. 46–57 (1977). `doi:10.1109/SFCS.1977.32`.

[13] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. The MIT Press, second edn. (2018).

[14] Sutton, R. S., Precup, D., and Singh, S. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, **112** (1999), 181. `doi:10.1016/S0004-3702(99)00052-1`.

[15] van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pp. 2094–2100 (2016).

[16] Watkins, C. J. C. H. and Dayan, P. Q-learning. *Machine Learning*, **8** (1992), 279. `doi:10.1007/BF00992698`.